Ember: A Compiler for Embedding Operations on Decoupled Access-Execute Architectures

Marco Siracusa[1-4], **Olivia Hsu**[2], Victor Soria[1], Joshua Randall[3], Arnaud Grasset[3], Eric Biscondi[3], Doug Joseph[4], Randy Allen[1], Fredrik Kjolstad[2], Miquel Moreto[1], Adrià Armejach[1]

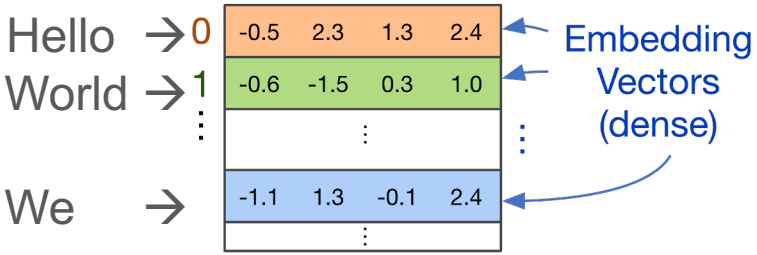
Barcelona Supercomputing Center
Centro Nacional de Supercomputación

Stanford | ENGINEERING
Computer Science

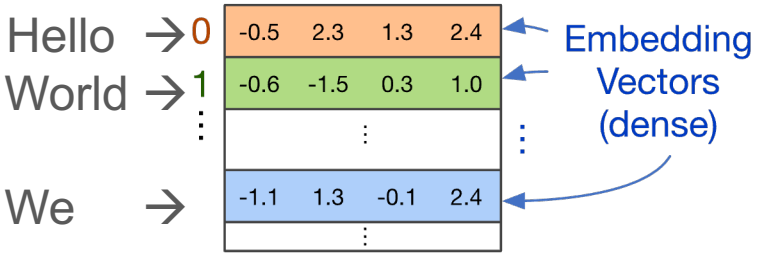
arm

SAMSUNG SAIT

Embedding operations are important for modern ML



Embedding operations are important for modern ML



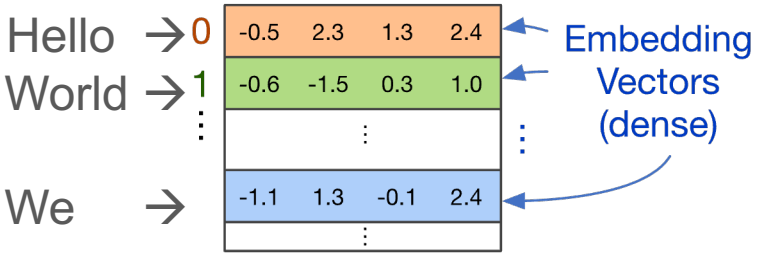
Example Sentences

"Hello world",
"We like tensors",
"compilers are pretty awesome too"

Converted to Indices

```
[      0,       1]
[      5, 113533, 133347]
[ 11937,    2116, 108171, 5906, 134887]
```

Embedding operations are important for modern ML



Example Sentences

"Hello world",
"We like tensors",
"compilers are pretty awesome too"

Converted to Indices

```
[      0,       1]
[      5, 113533, 133347]
[ 11937,    2116, 108171, 5906, 134887]
```

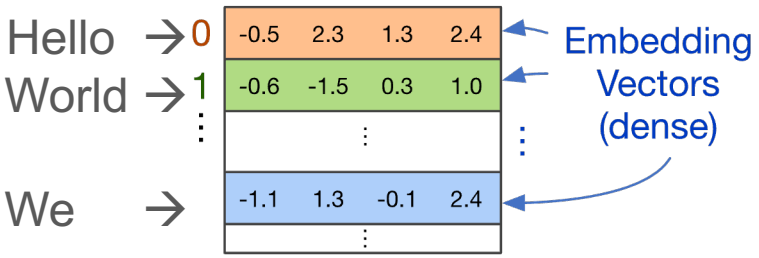
Indices in a Compressed Format

Segments: `[ 0, 2, 7, 12]`

Pointer Tuples (beg,end): `[(0, 2), (2,7), (7, 12)]`

pos:	0	1	2	3	…	6	7	…	11
idxs:	0,	1,	5,	152947,	…,	133347,	11937,	…,	134887]

Embedding operations are important for modern ML



Example Sentences

"Hello world",
"We like tensors",
"compilers are pretty awesome too"

Converted to Indices

```
[      0,       1]
[      5, 113533, 133347]
[ 11937,    2116, 108171, 5906, 134887]
```

Indices in a Compressed Format

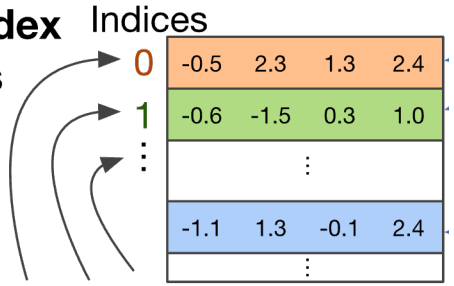
Segments:[0, 2, 7, 12]

Pointer Tuples (beg,end):[(0, 2), (2,7), (7, 12)]

pos: 0 1 2 3 … 6 7 … 11

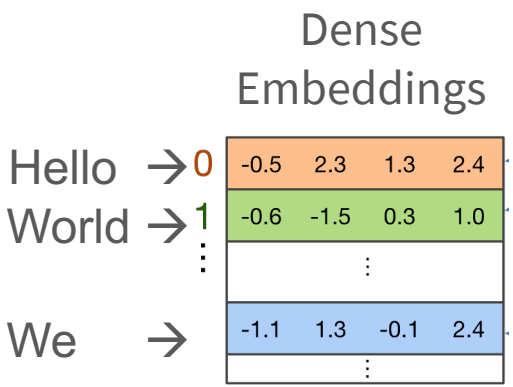
idxs:[0, 1, 5, 152947, …, 133347, 11937, …, 134887]

Irregular Index Lookups



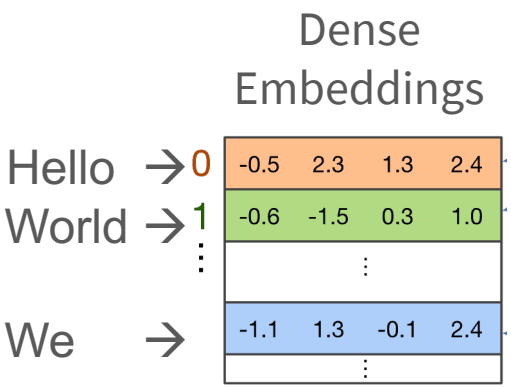
idxs:[0, 1, 5, . . .]

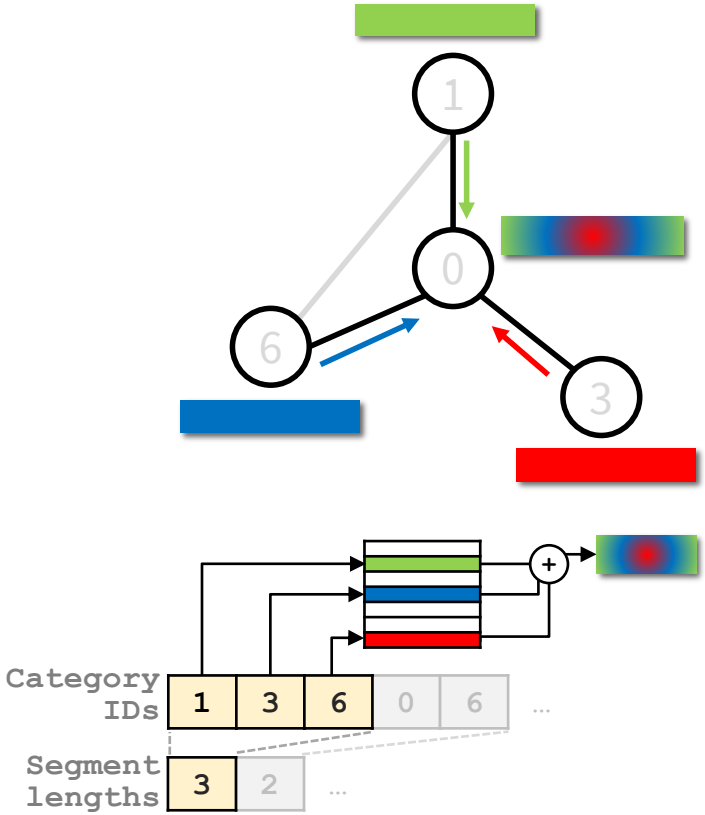
Many types of embedding lookups



Text Embeddings

Many types of embedding lookups

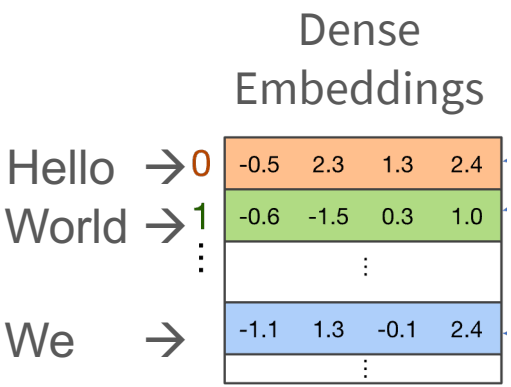


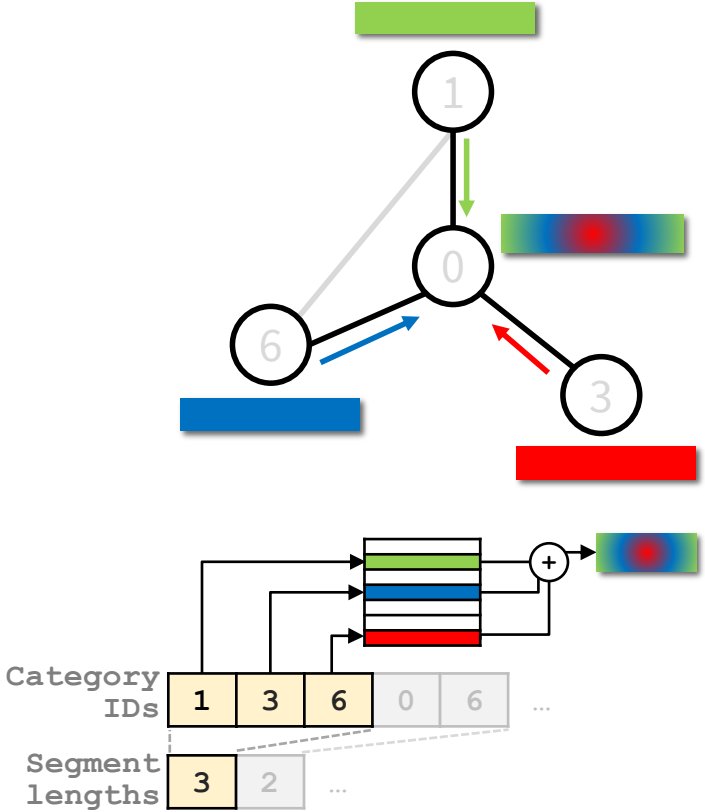


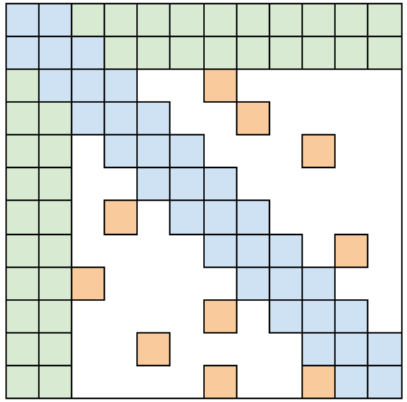
Text Embeddings

Graph Learning

Many types of embedding lookups





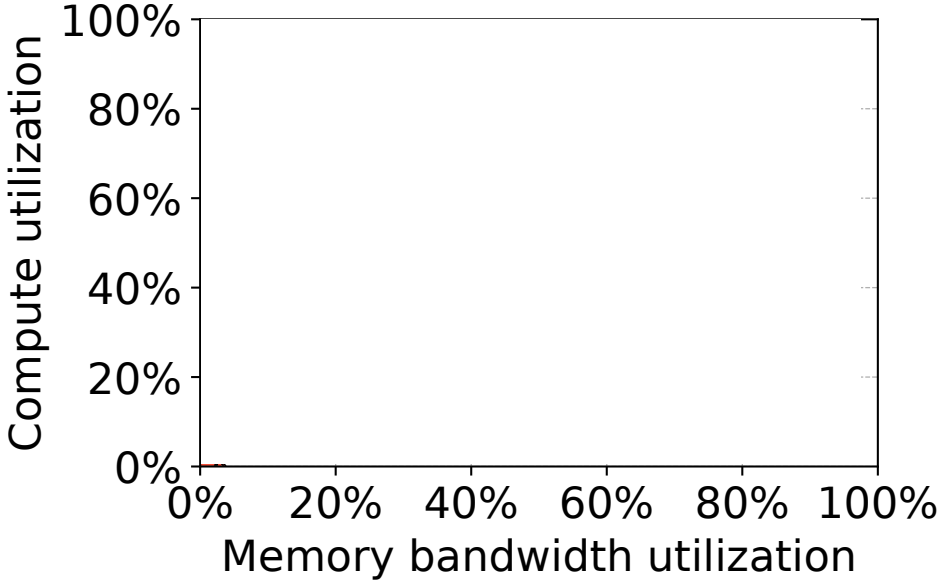


Text Embeddings

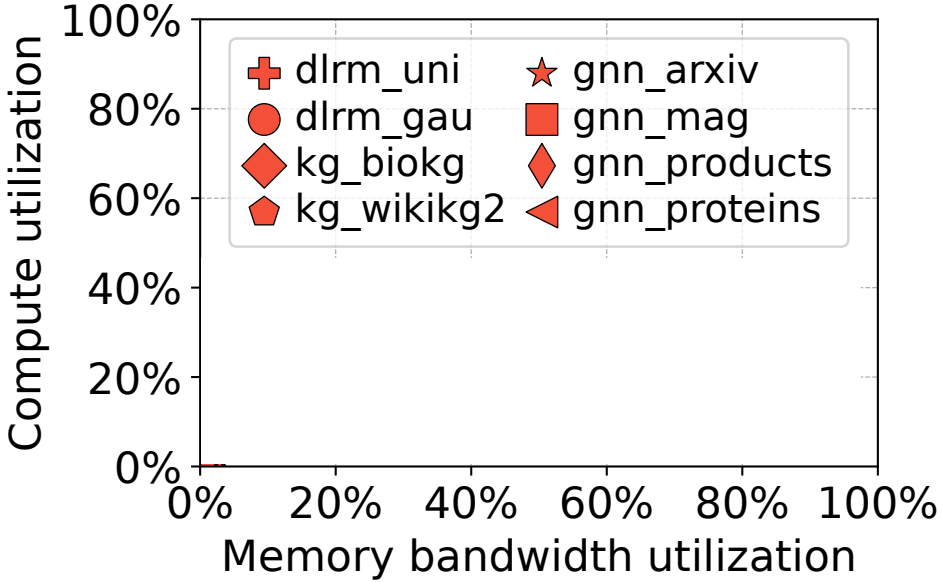
Graph Learning

Sparse Attention

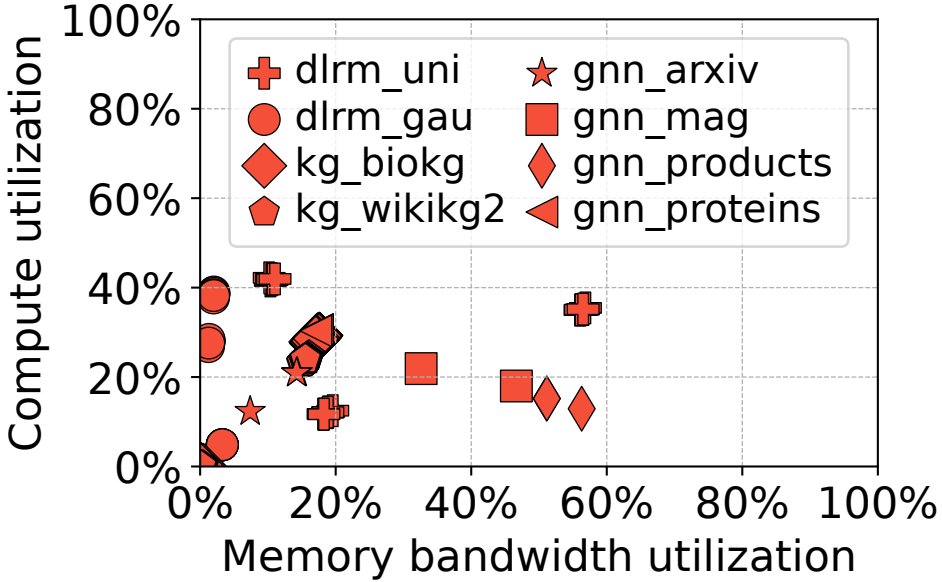
Challenges with traditional architectures



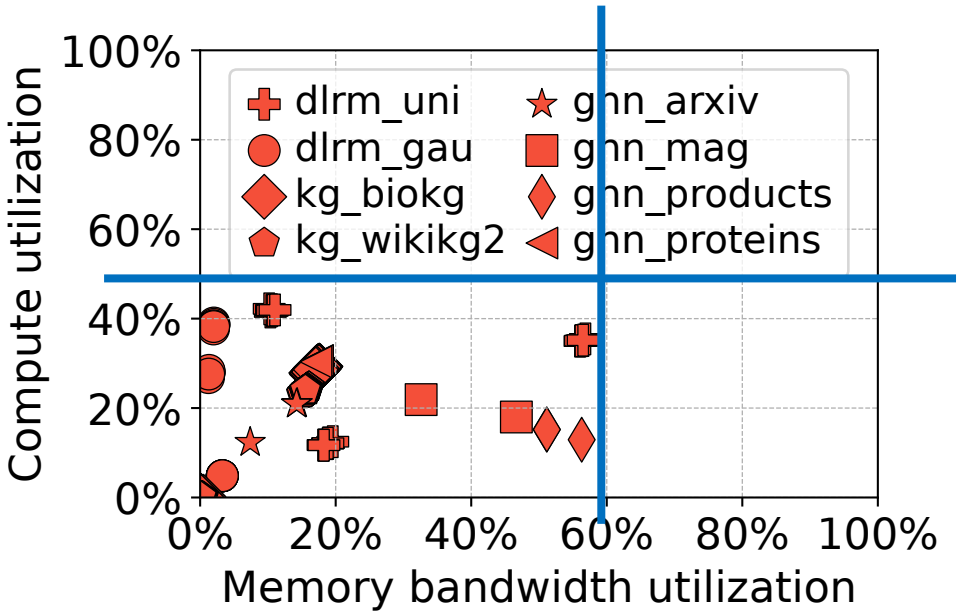
Challenges with traditional architectures



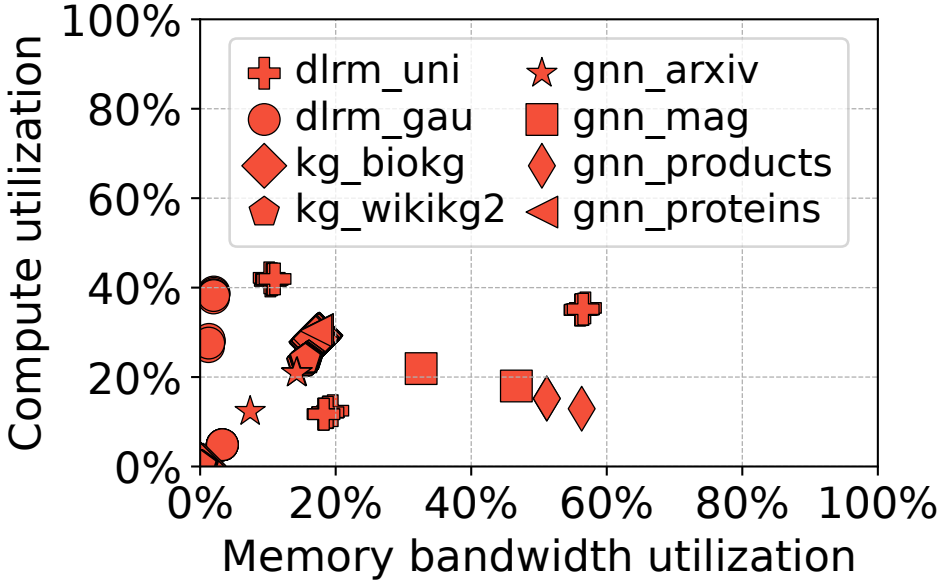
Challenges with traditional architectures

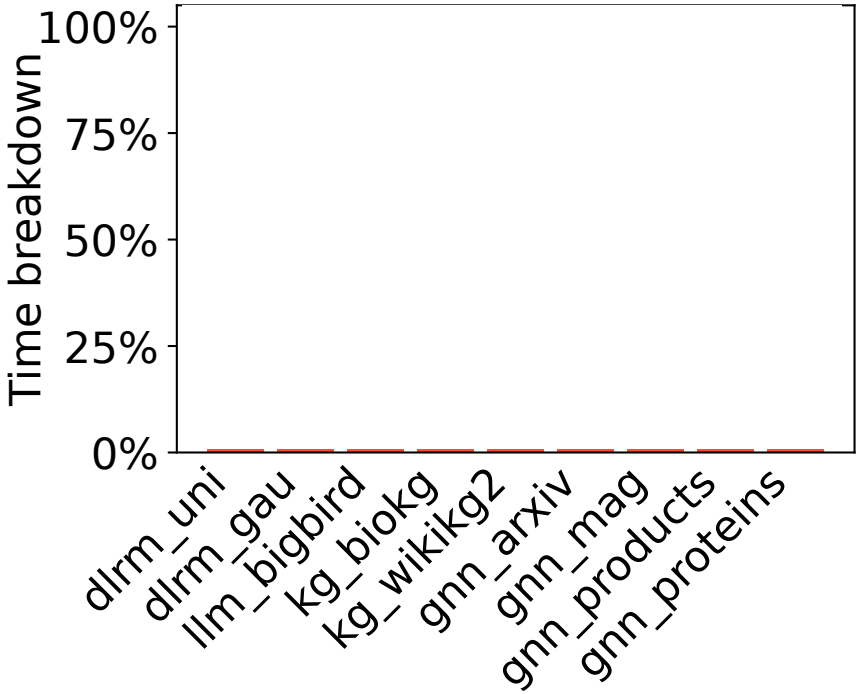


Challenges with traditional architectures

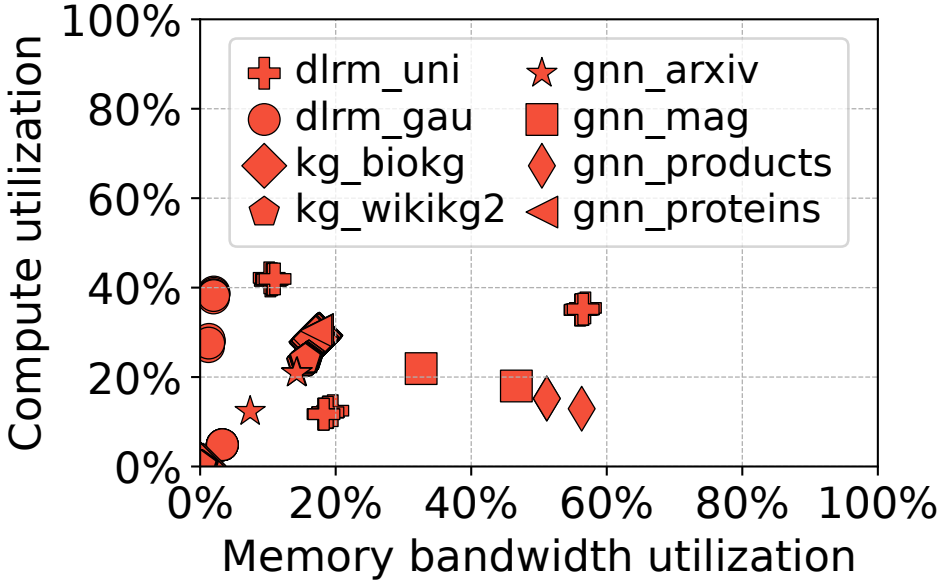


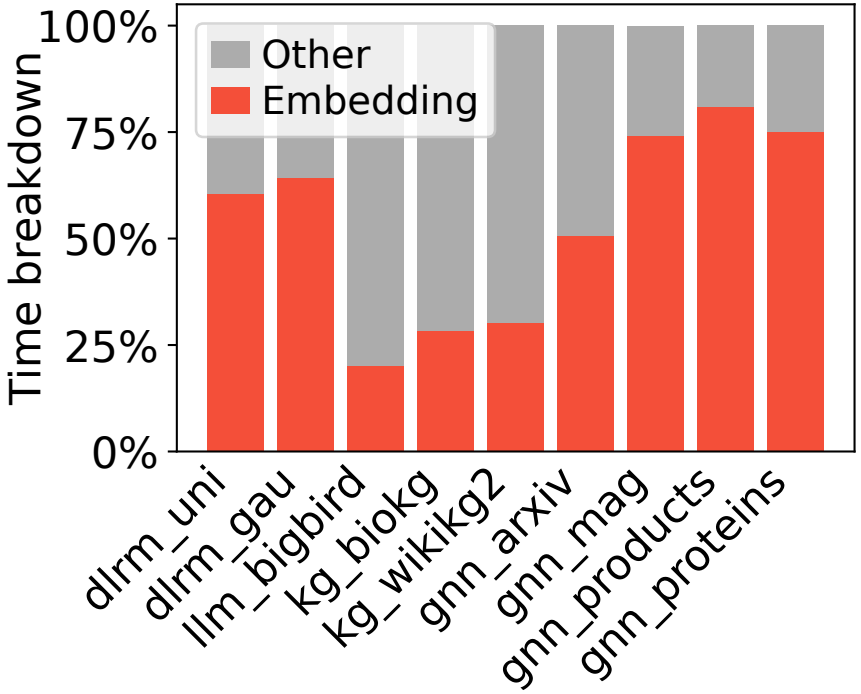
Challenges with traditional architectures



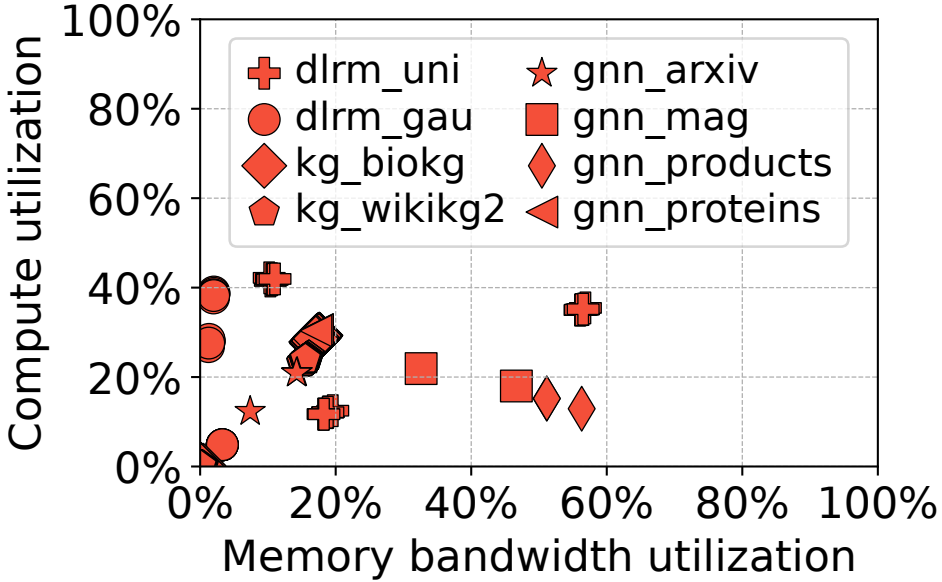


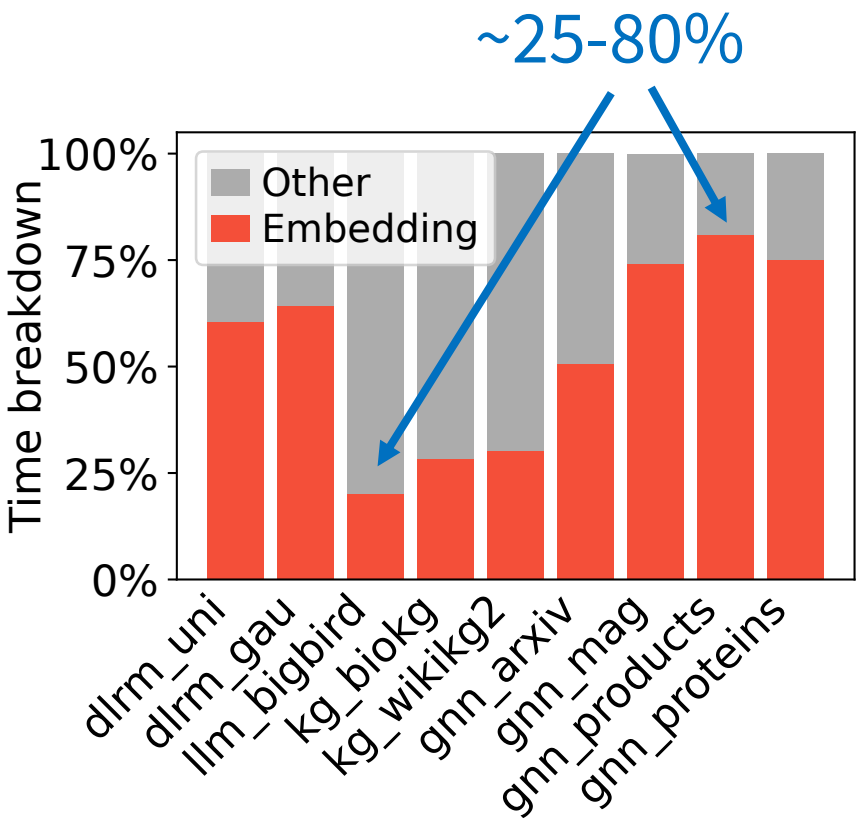
Challenges with traditional architectures





Challenges with traditional architectures



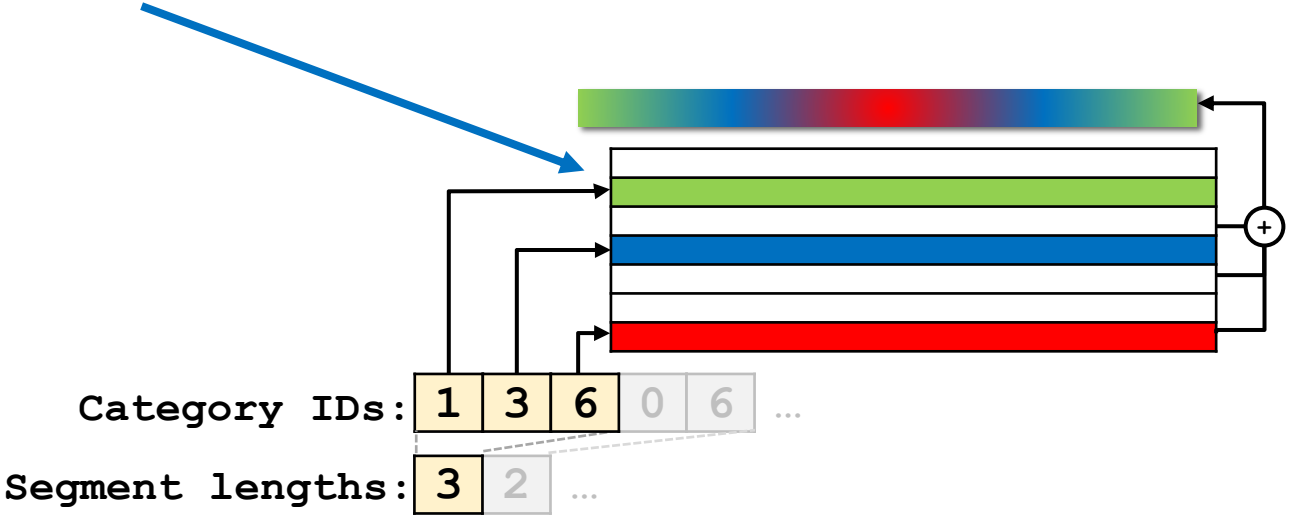


It’s the memory-level parallelism

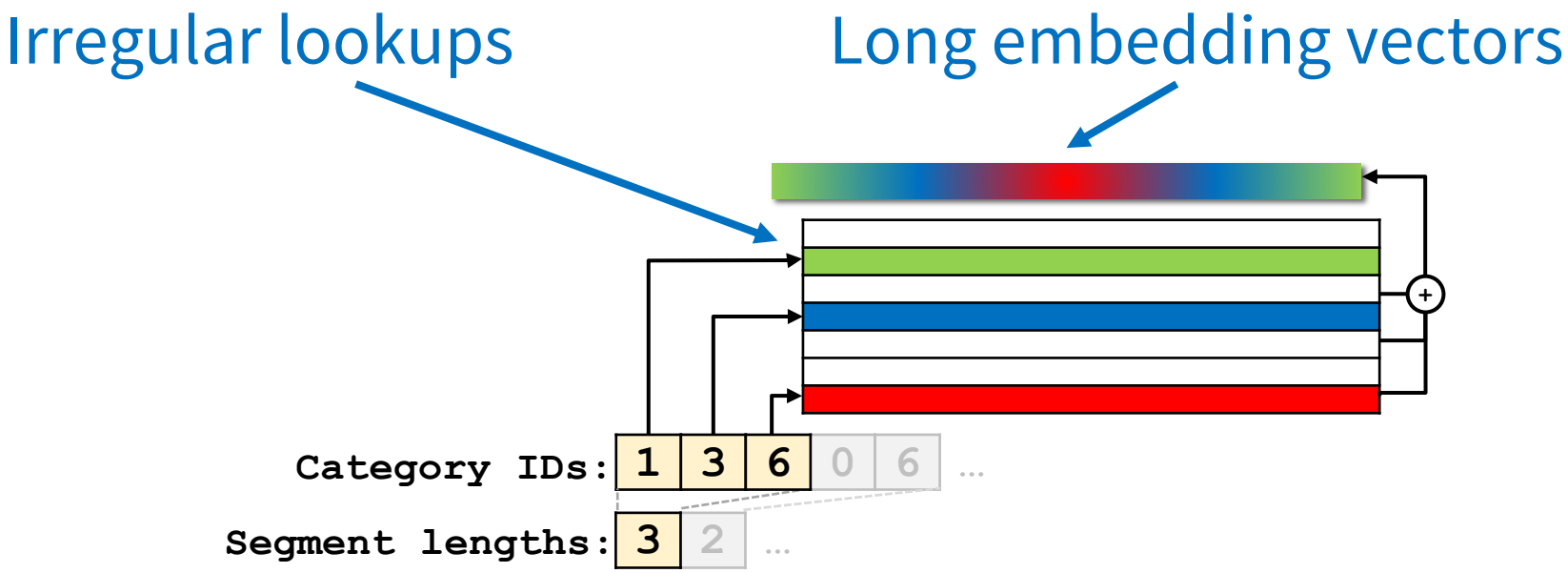
Challenges with traditional architectures

Embedding operations have irregular properties

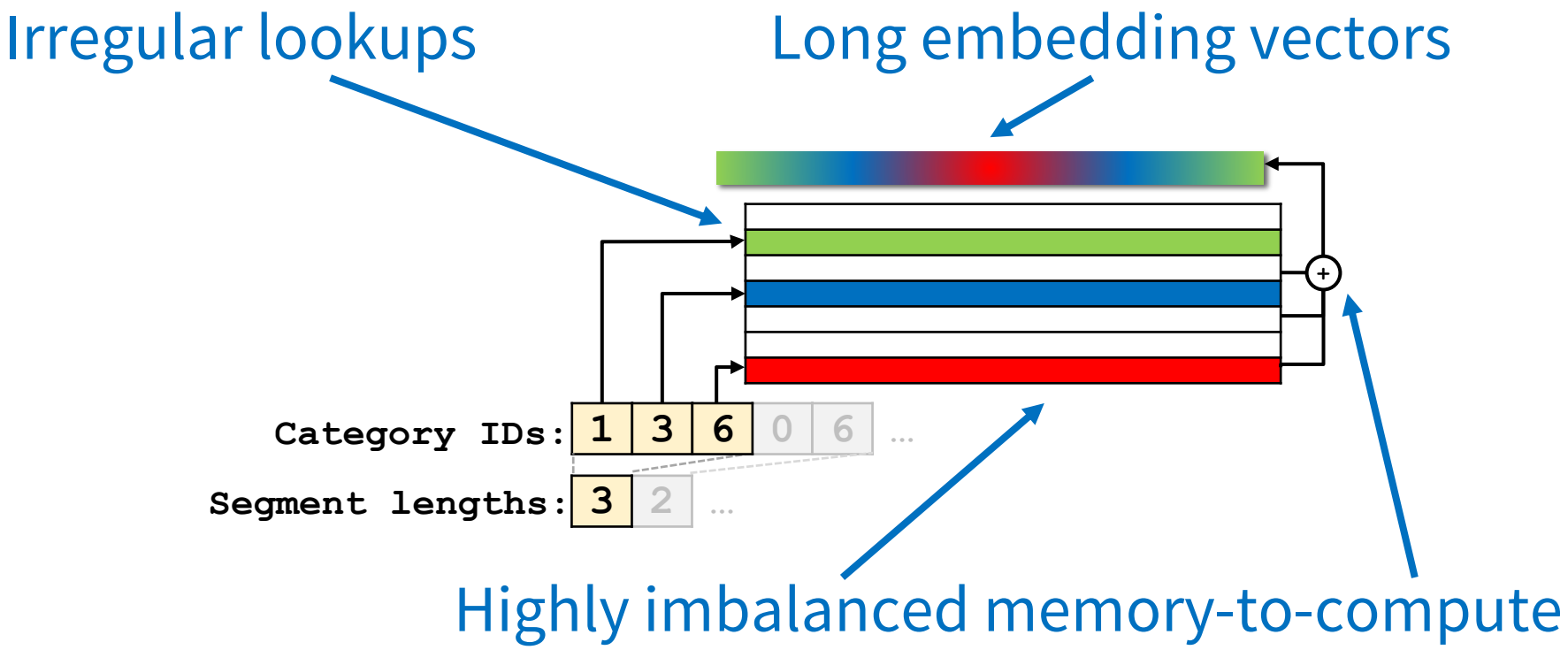
Irregular lookups



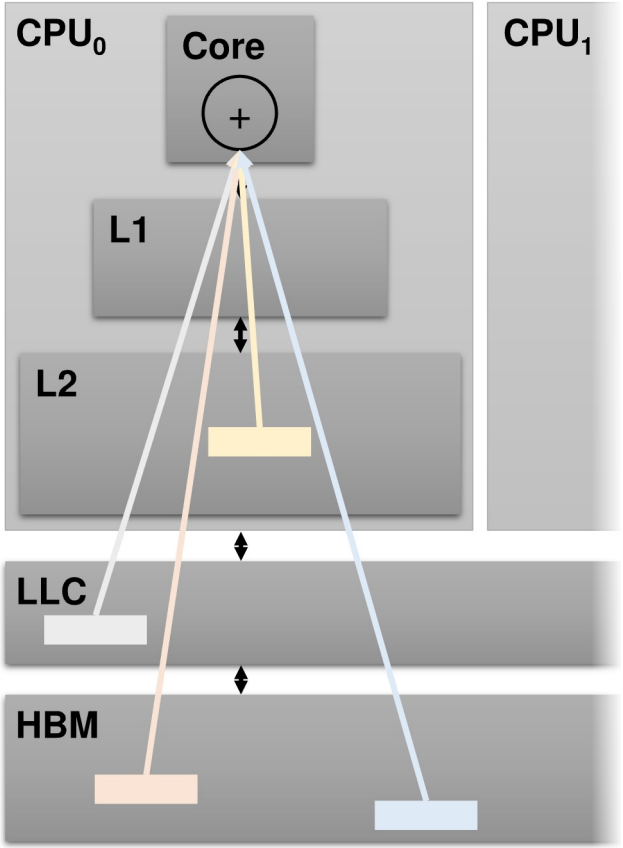
Embedding operations have irregular properties



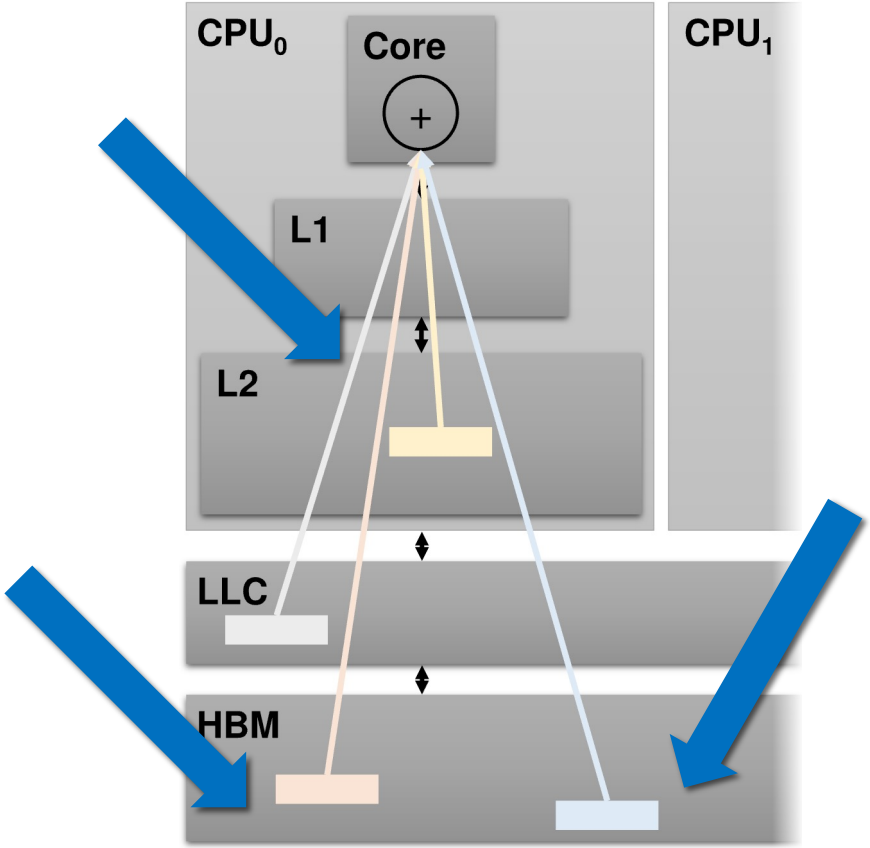
Embedding operations have irregular properties



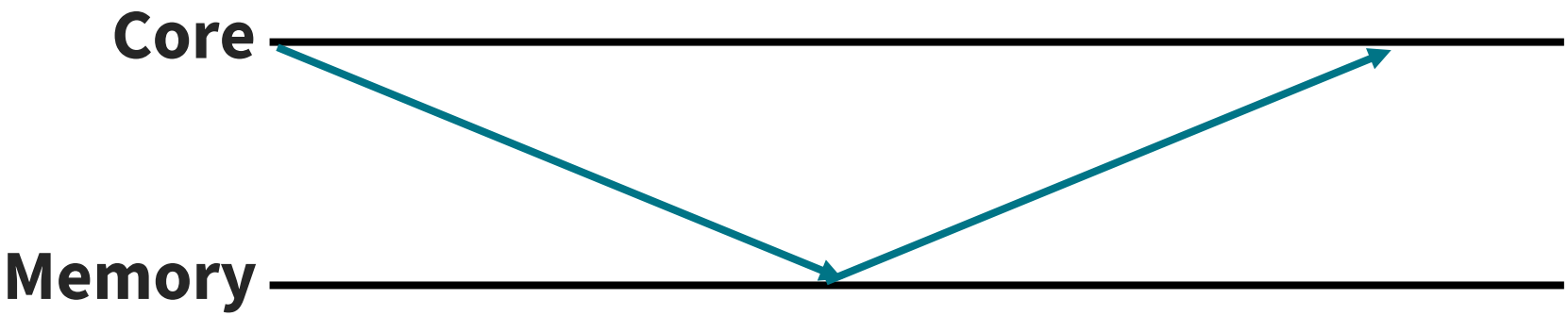
Consider a traditional CPU architecture



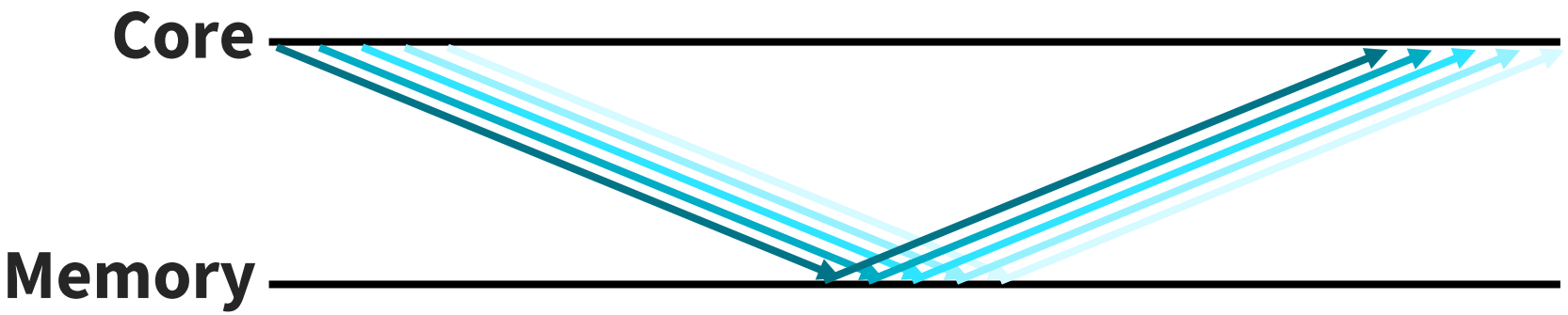
Consider a traditional CPU architecture



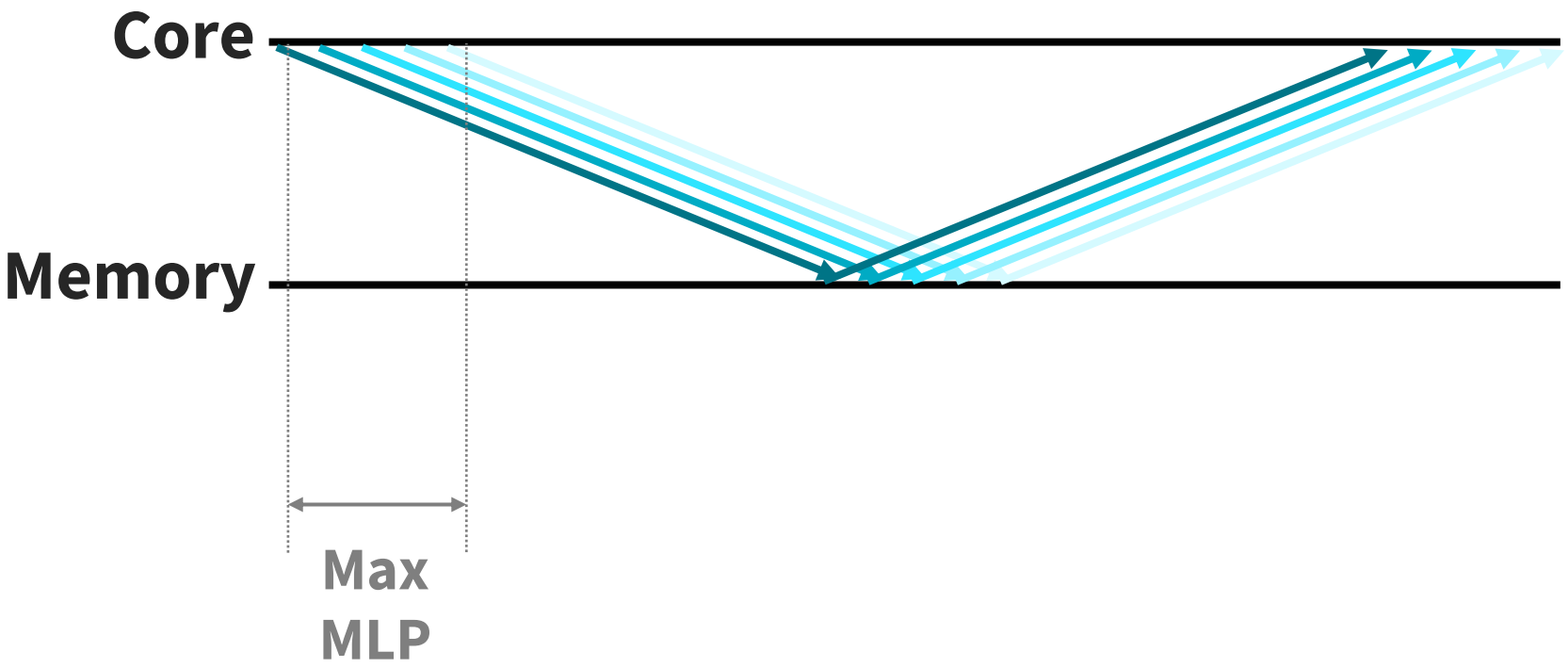
Memory latency, the cause of poor performance



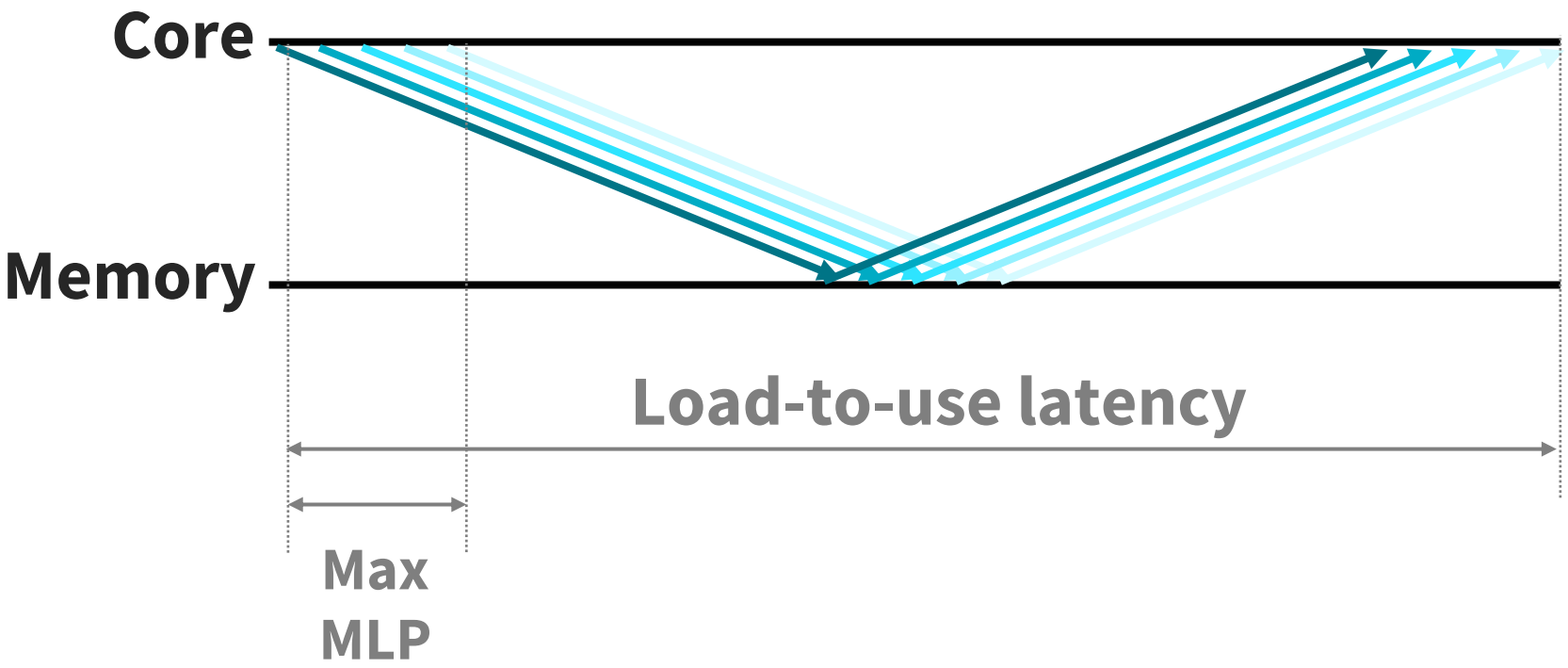
Memory latency, the cause of poor performance



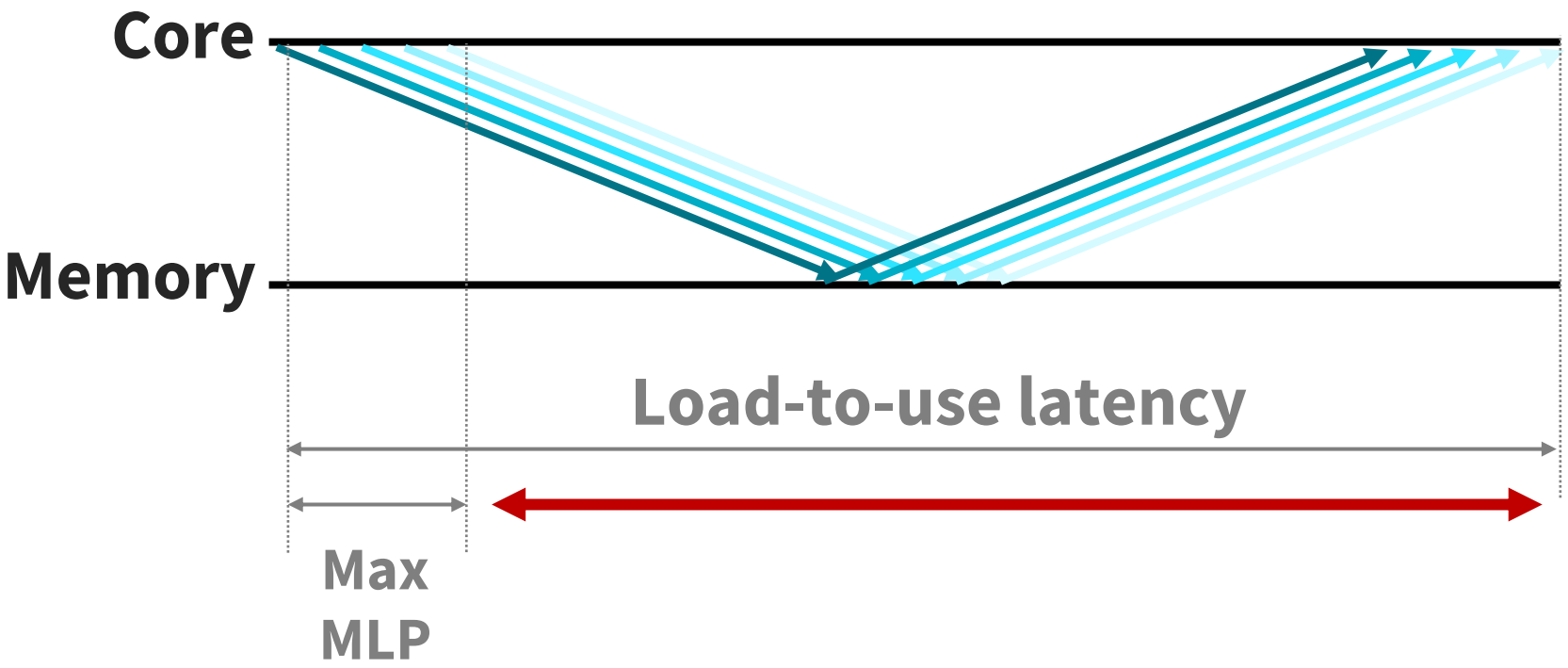
Memory latency, the cause of poor performance



Memory latency, the cause of poor performance



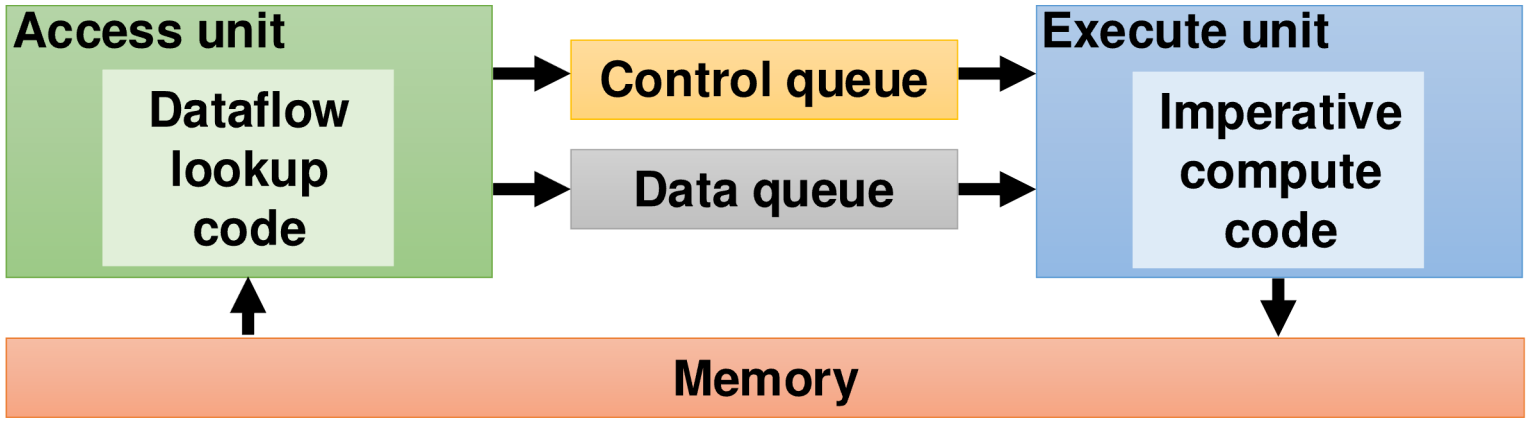
Memory latency, the cause of poor performance



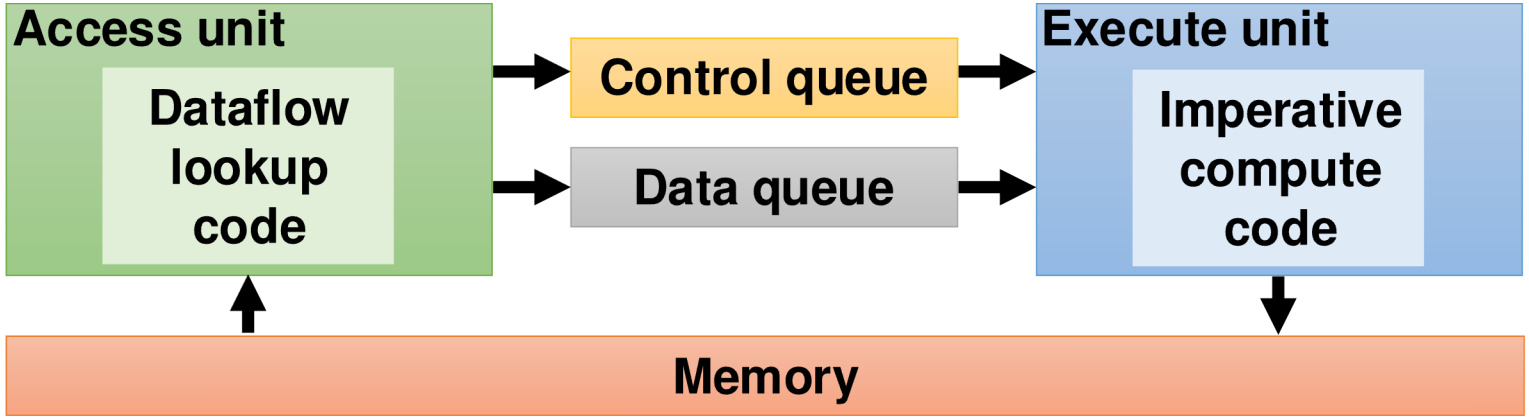
Solution:
Decoupled Access-Execute (DAE) designs

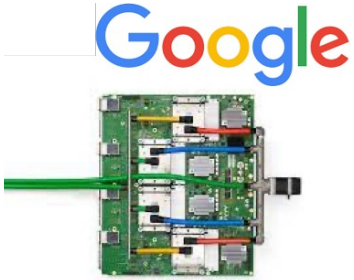
and the Tensor Marshaling Unit (TMU)

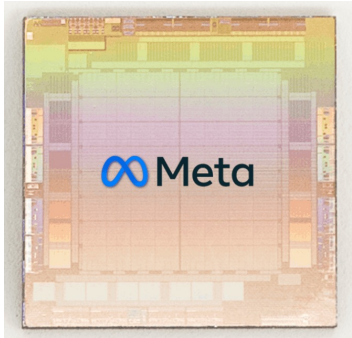
Decoupling access and execute

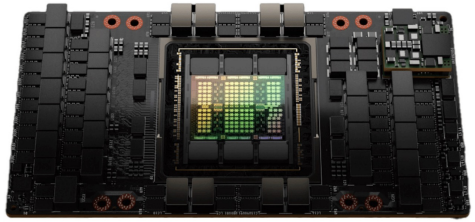


Decoupling access and execute, increasingly important

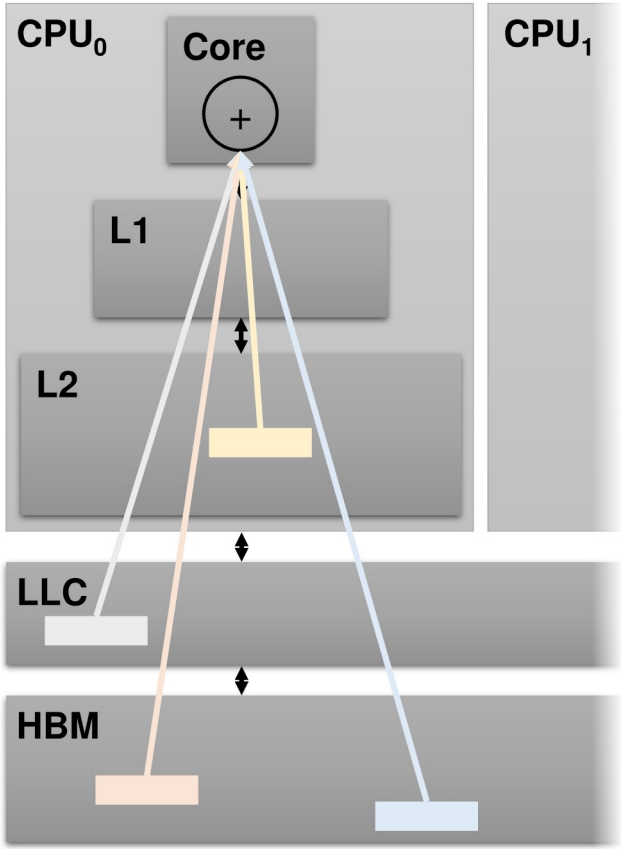




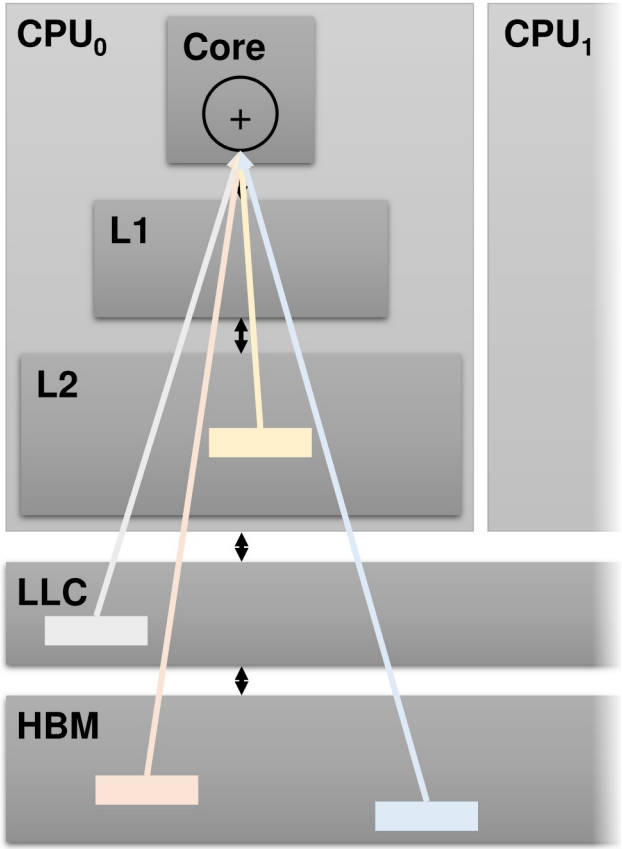


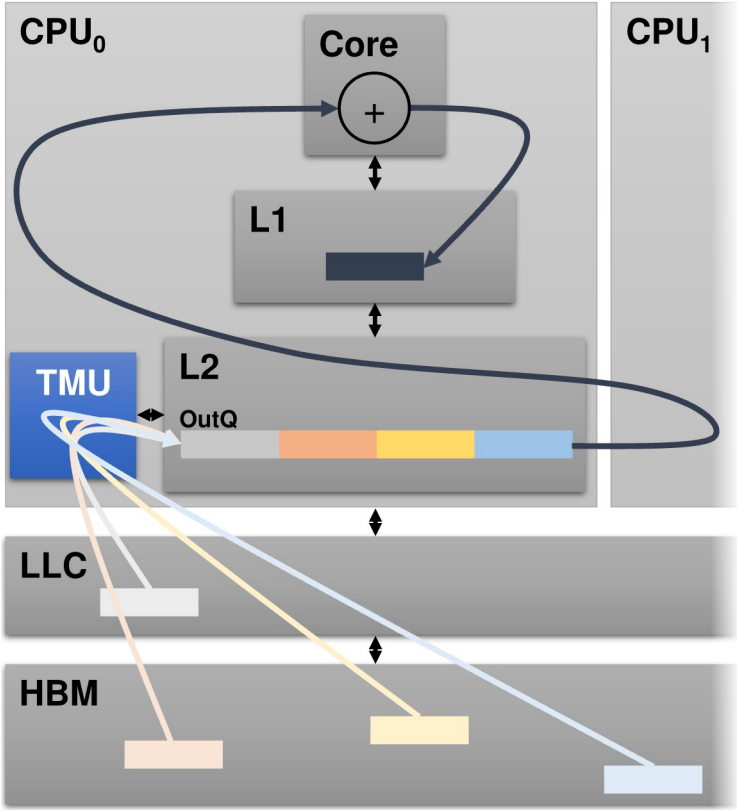


Our exact DAE processor backend



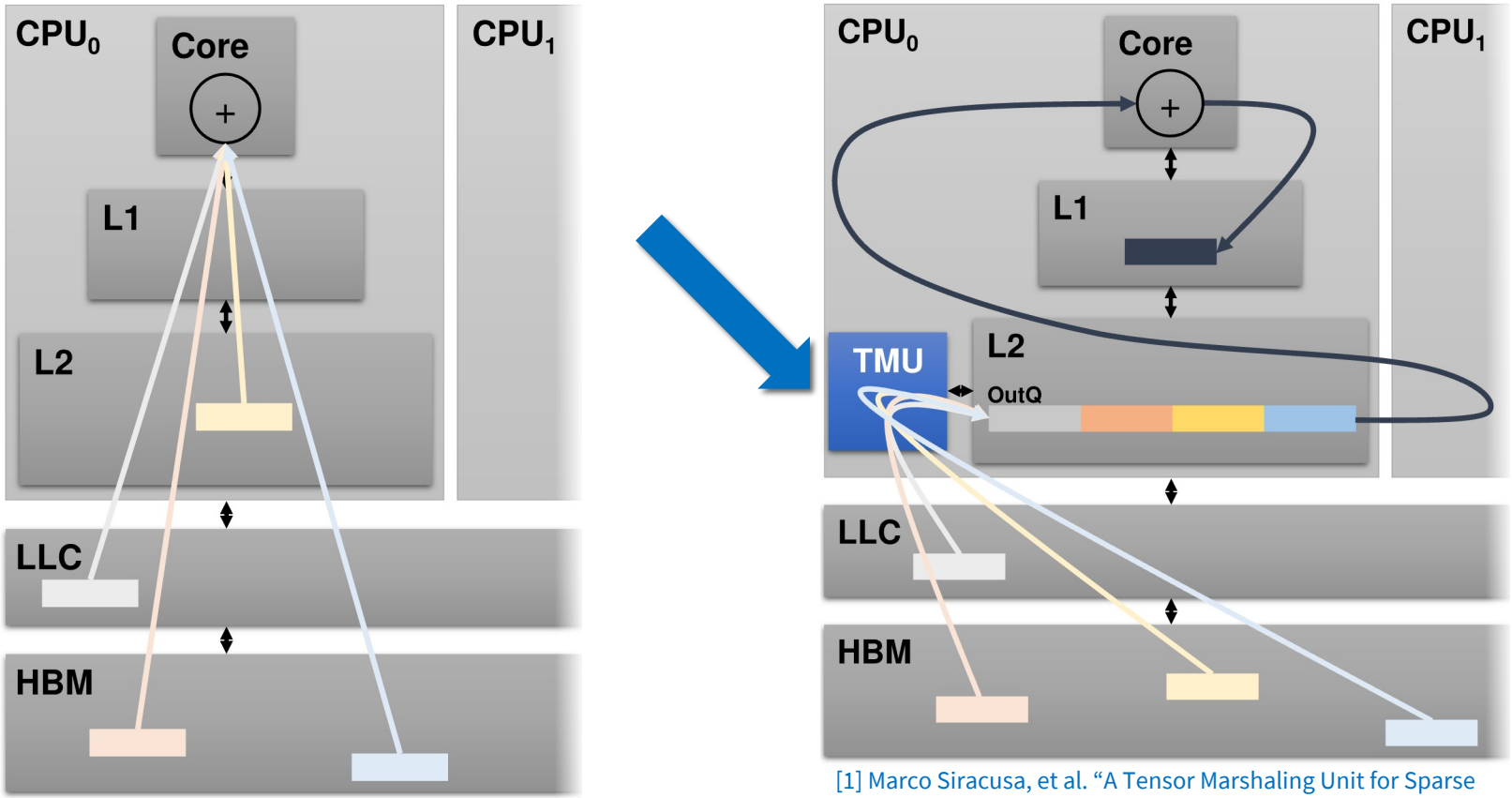
Our exact DAE processor backend





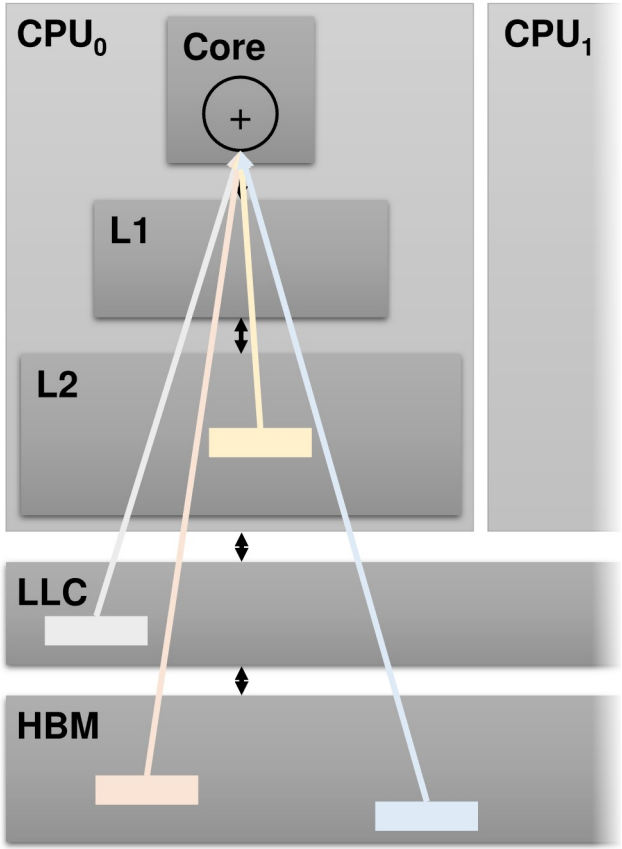
[1] Marco Siracusa, et al. “A Tensor Marshaling Unit for Sparse Tensor Algebra on General-Purpose Processors”. MICRO 2023

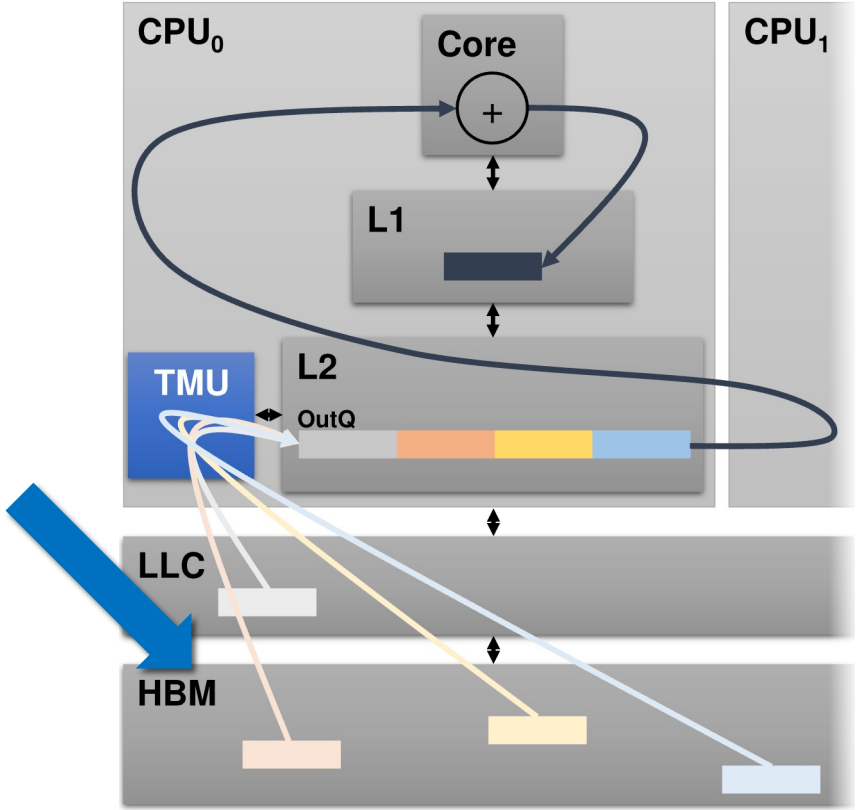
Our exact DAE processor backend



[1] Marco Siracusa, et al. “A Tensor Marshaling Unit for Sparse Tensor Algebra on General-Purpose Processors”. MICRO 2023

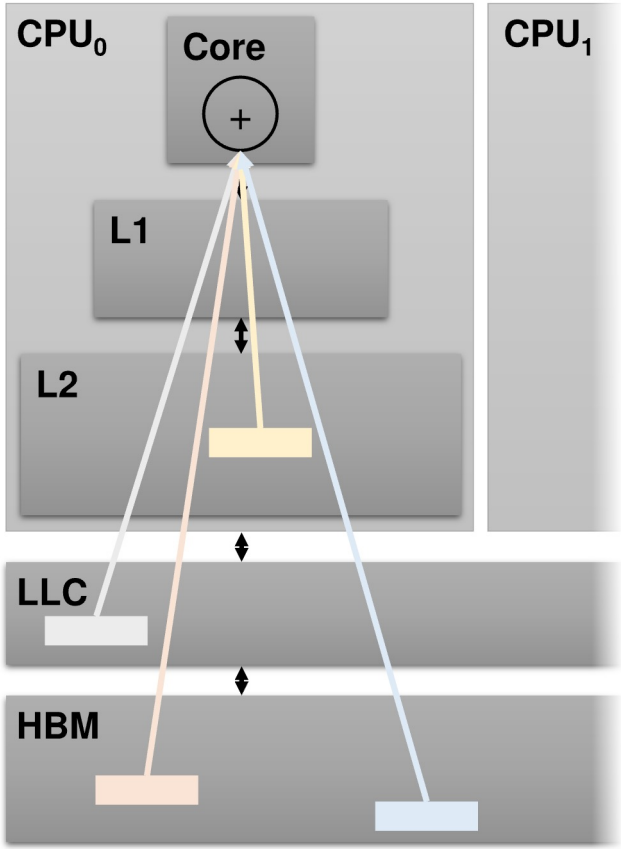
Our exact DAE processor backend

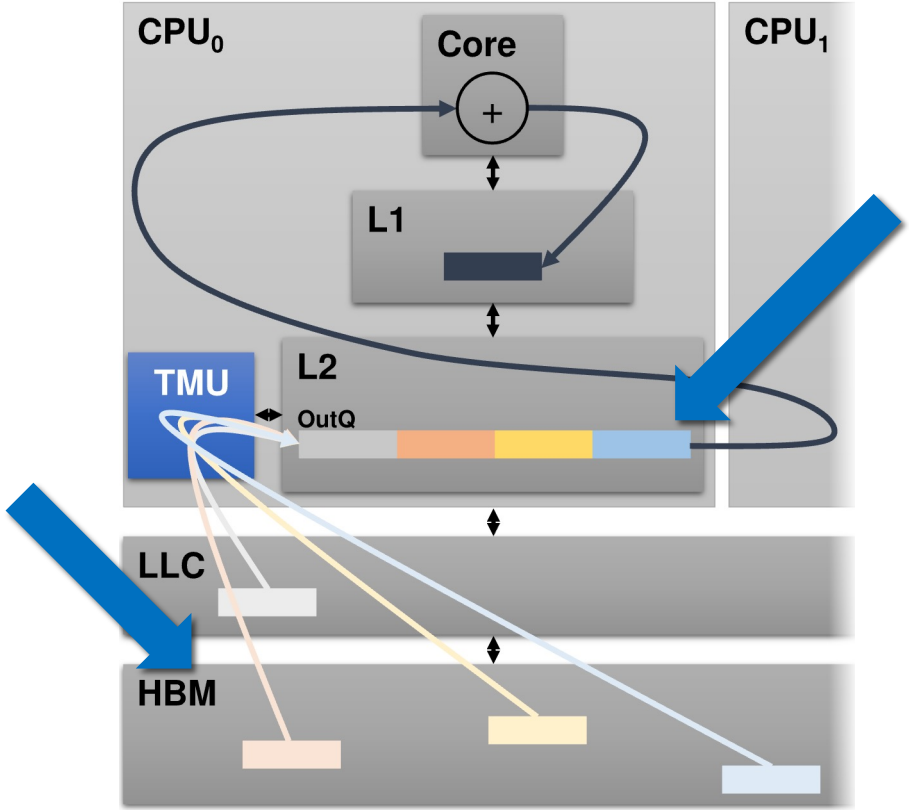




[1] Marco Siracusa, et al. “A Tensor Marshaling Unit for Sparse Tensor Algebra on General-Purpose Processors”. MICRO 2023

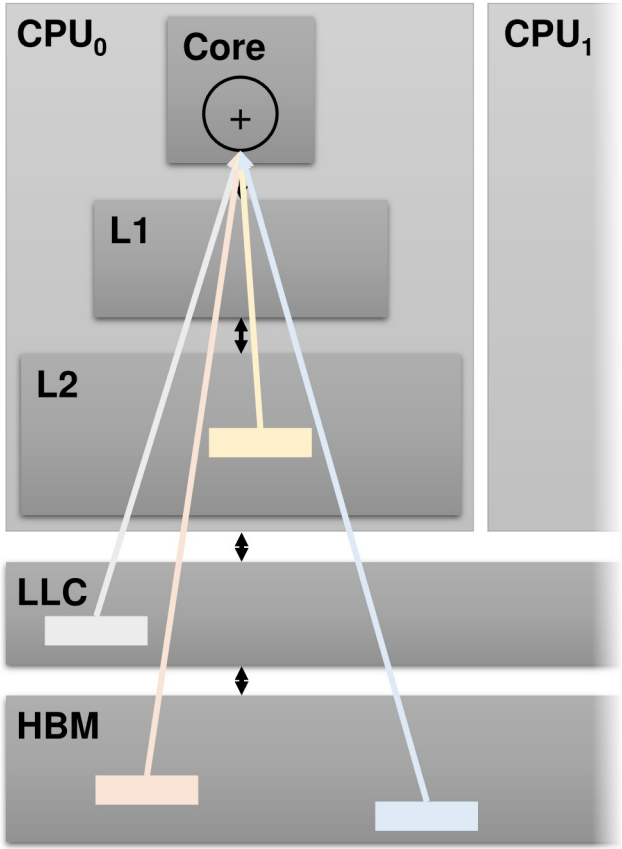
Our exact DAE processor backend

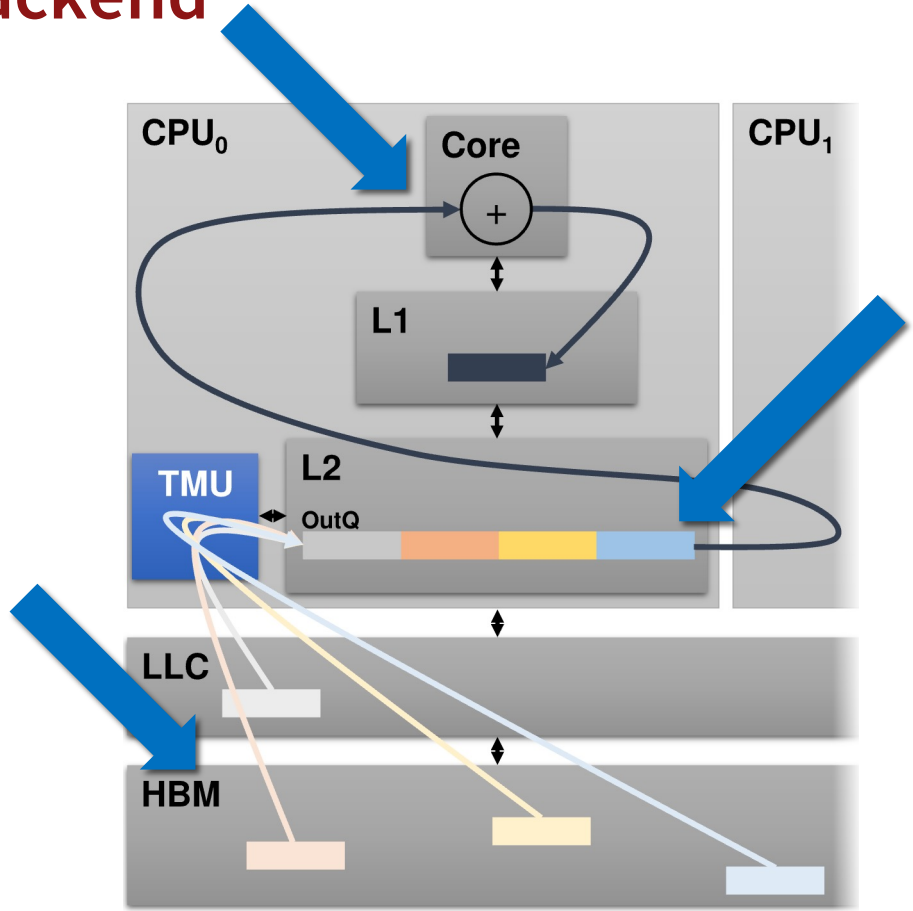




[1] Marco Siracusa, et al. “A Tensor Marshaling Unit for Sparse Tensor Algebra on General-Purpose Processors”. MICRO 2023

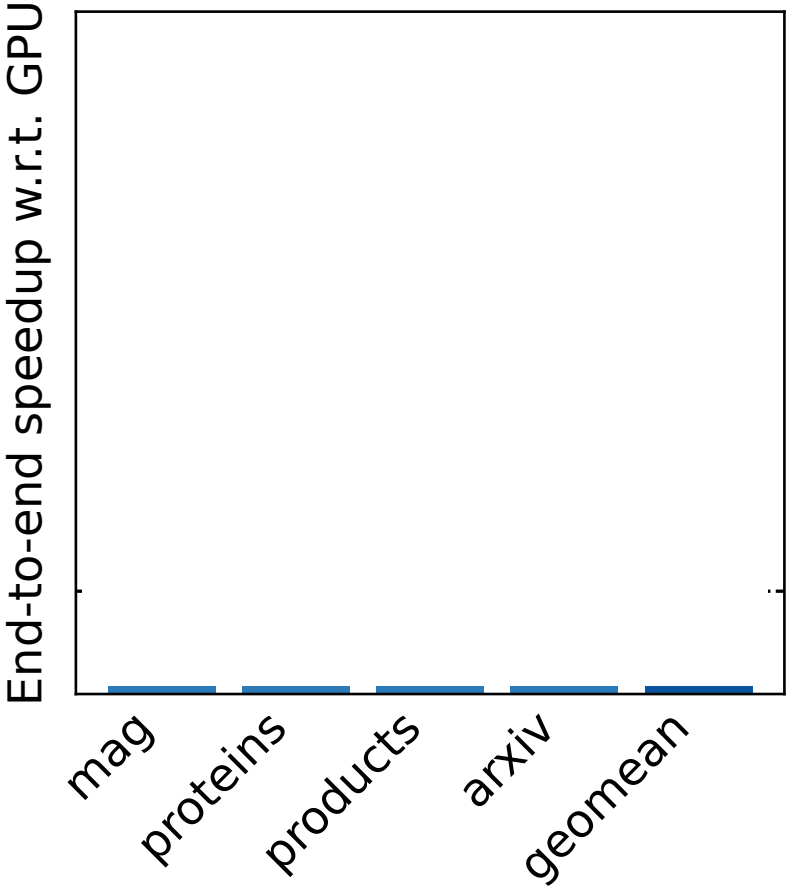
Our exact DAE processor backend



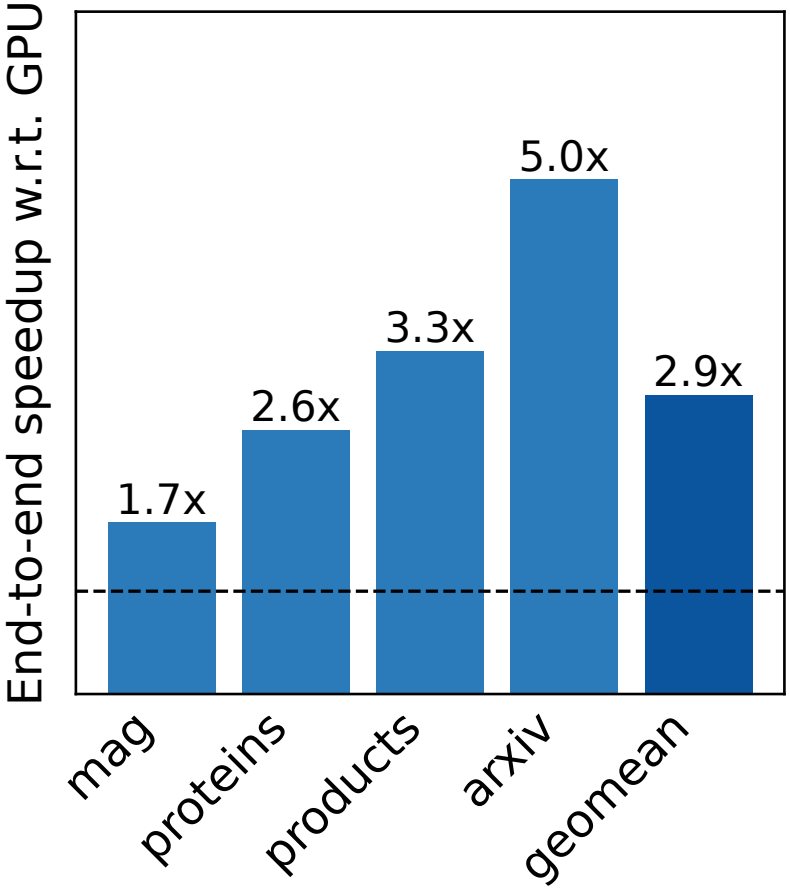


[1] Marco Siracusa, et al. “A Tensor Marshaling Unit for Sparse Tensor Algebra on General-Purpose Processors”. MICRO 2023

The potential of a DAE processor versus GPUs



The potential of a DAE processor versus GPUs



However, low-level decoupled code is hard to write

TMU code:

```
…
i_tr = loop_tr(0, num_idxs)
i_str = i_tr.mem_str(idx_vec, [i_tr.ite])
e_tr = loop_tr(0, emb_len)
e_str = i_tr.mem_str(emb_tbl, [i_str, e_tr.ite])
serialize(i_tr.ite, e_tr.ite, e_str)
```

CPU code:

```
…
while(…)
  deserialize(i, j, v)
  compute(i, j, v, res)
```

However, low-level decoupled code is hard to write

TMU code:

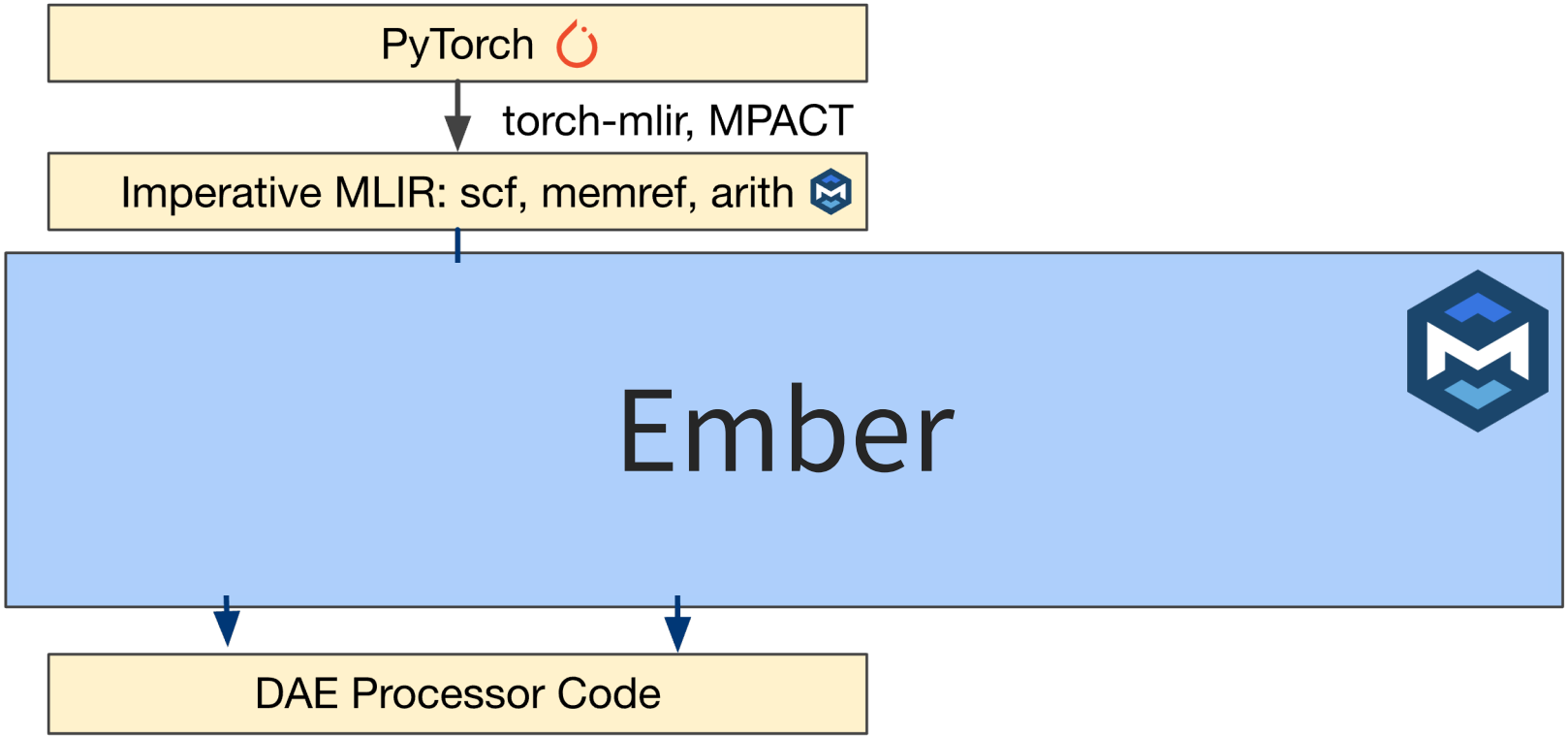
```
…
i_tr = loop_tr(0, num_idxs)
i_str = i_tr.mem_str(idx_vec, [i_tr.ite])
e_tr = loop_tr(0, emb_len)
e_str = i_tr.mem_str(emb_tbl, [i_str, e_tr.ite])
serialize(i_tr.ite, e_tr.ite, e_str)
```

CPU code:

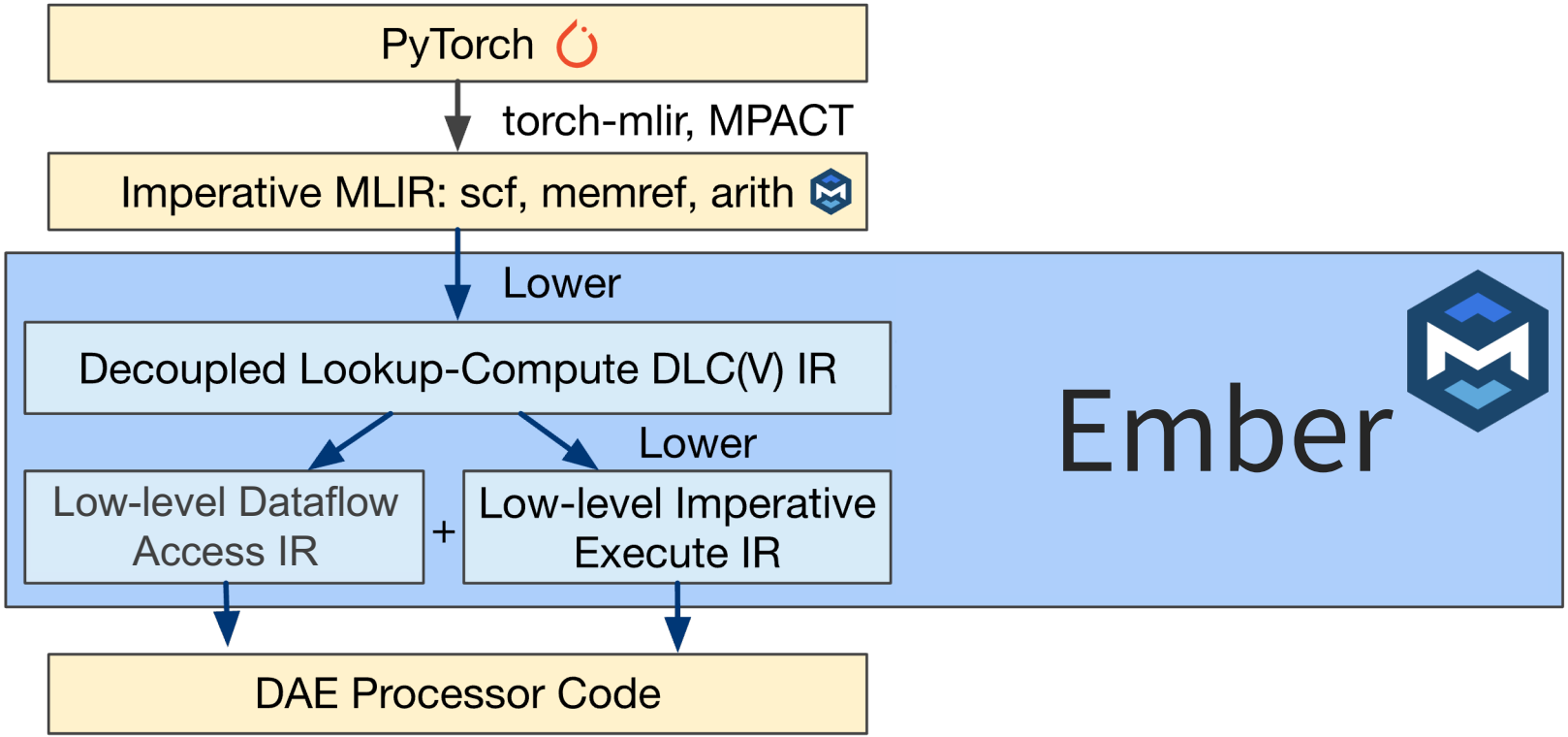
```
…
while(…)
  deserialize(i, j, v)
  compute(i, j, v, res)
```

This is just for one model class, need to rewrite this for all types of embedding operations

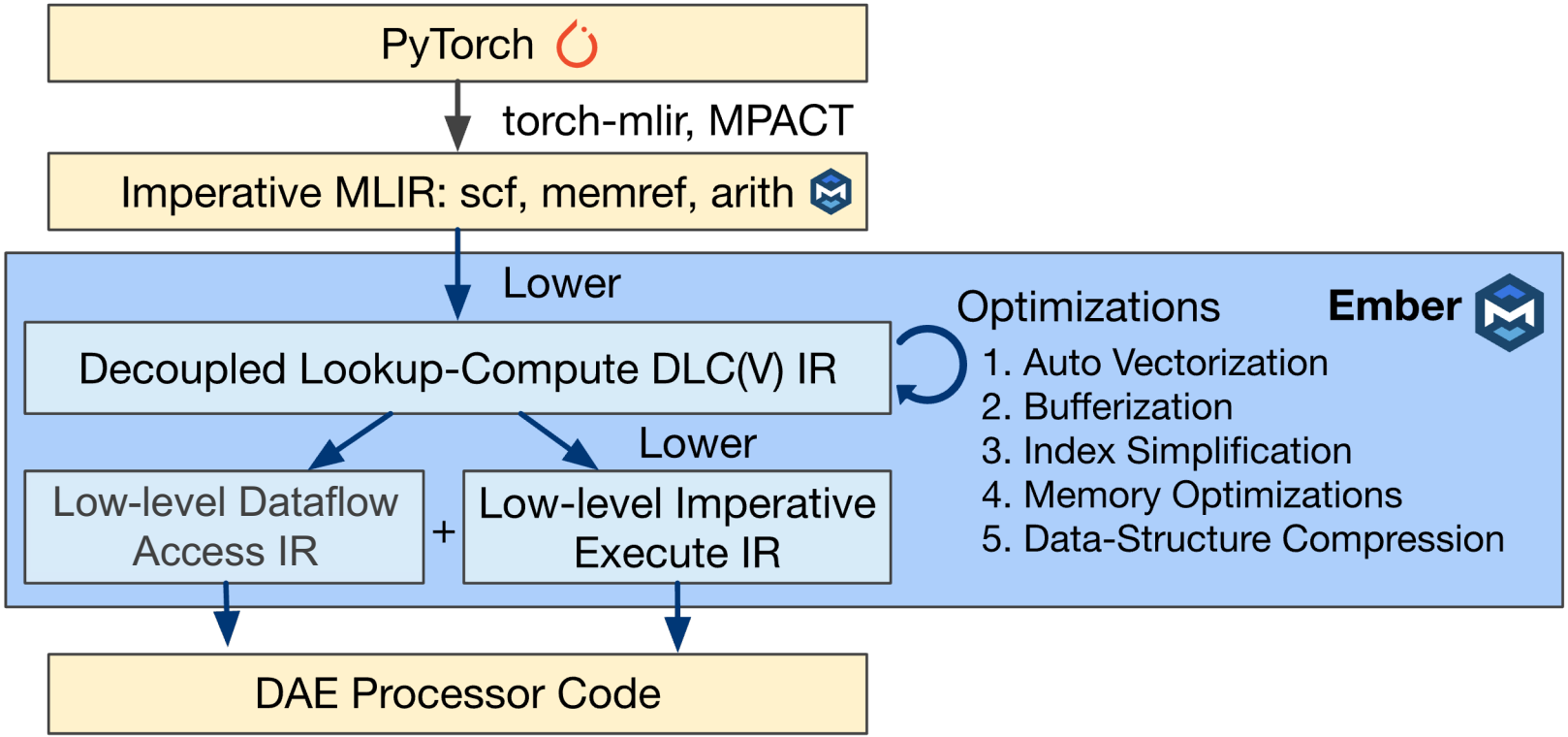
The Ember compiler



The Ember compiler



The Ember compiler



The Decoupled Lookup-Compute (DLC) IR

Any CPU Statement	*STMT*	*Memory Reference*	*mref*
CPU variable	*var*	*Stream Variable*	*s*
Unsigned Integer	uint		

For-Loop	*FOR*	::=	for (*HEAD*) { *SDEC** *BODY* }
For Header	*HEAD*	::=	*s* = *r* to *r* step uint
Stream Declaration	*SDEC*	::=	*s* = *se*
Loop Body	*BODY*	::=	*CALL* | *FOR*
Callback	*CALL*	::=	callback { *TVAL** *STMT*$^+$ }
Value Conversion	*TVAL*	::=	*var* = to_val(*s*)
Loop Range	*r*	::=	*s* | *var* | uint
Stream Expression	*se*	::=	*ms* | *as*
Memory Stream	*ms*	::=	mem_str(*mref*[*indices**])
Indices	*indices*	::=	*s* | *var* | uint
ALU Stream	*as*	::=	alu_str(*sop*, s_1, s_2)
Stream Ops	*sop*	::=	+ | − | ∗ | / | % | ...

DLC IR for Running Knowledge Graph (KG) example:

```
...
dlc.for(str<i32> s_i = 0 to num_idxs) {
  str<i32> s_idx = dlc.mem_str(idx_vec[i])
  dlc.for(str<i32> s_j = 0 to emb_len) {
    str<f32> s_v = dlc.mem_str(emb_tbl[idx,j])
    dlc.callback() {
      f32 i = dlc.to_val(s_i)
      f32 j = dlc.to_val(s_j)
      f32 v = dlc.to_val(s_v)
      compute(i, j, v, res) }}}
```

The Decoupled Lookup-Compute (DLC) IR

Any CPU Statement	*STMT*	*Memory Reference*	*mref*
CPU variable	*var*	*Stream Variable*	*s*
Unsigned Integer	uint		

Control code

For-Loop	*FOR*	::=	for (*HEAD*) { *SDEC** *BODY* }
For Header	*HEAD*	::=	*s* = *r* to *r* step uint
Stream Declaration	*SDEC*	::=	*s* = *se*
Loop Body	*BODY*	::=	*CALL* | *FOR*
Callback	*CALL*	::=	callback { *TVAL** *STMT*$^+$ }
Value Conversion	*TVAL*	::=	*var* = to_val(*s*)
Loop Range	*r*	::=	*s* | *var* | uint
Stream Expression	*se*	::=	*ms* | *as*
Memory Stream	*ms*	::=	mem_str(*mref*[*indices**])
Indices	*indices*	::=	*s* | *var* | uint
ALU Stream	*as*	::=	alu_str(*sop*, s_1, s_2)
Stream Ops	*sop*	::=	+ | − | ∗ | / | % | ...

DLC IR for Running Knowledge Graph (KG) example:

```
...
dlc.for(str<i32> s_i = 0 to num_idxs) {
  str<i32> s_idx = dlc.mem_str(idx_vec[i])
  dlc.for(str<i32> s_j = 0 to emb_len) {
    str<f32> s_v = dlc.mem_str(emb_tbl[idx,j])
    dlc.callback() {
      f32 i = dlc.to_val(s_i)
      f32 j = dlc.to_val(s_j)
      f32 v = dlc.to_val(s_v)
      compute(i, j, v, res) }}}
```

The Decoupled Lookup-Compute (DLC) IR

Any CPU Statement	*STMT*	*Memory Reference*	*mref*
CPU variable	*var*	*Stream Variable*	*s*
Unsigned Integer	uint		

Control code	*For-Loop*	*FOR*	::=	for (*HEAD*) { *SDEC** *BODY* }
	For Header	*HEAD*	::=	*s* = *r* to *r* step uint
	Stream Declaration	*SDEC*	::=	*s* = *se*
	Loop Body	*BODY*	::=	*CALL* | *FOR*
	Callback	*CALL*	::=	callback { *TVAL** *STMT*$^+$ }
	Value Conversion	*TVAL*	::=	*var* = to_val(*s*)
	Loop Range	*r*	::=	*s* | *var* | uint
	Stream Expression	*se*	::=	*ms* | *as*
	Memory Stream	*ms*	::=	mem_str(*mref*[*indices**])
Access code	*Indices*	*indices*	::=	*s* | *var* | uint
	ALU Stream	*as*	::=	alu_str(*sop*, s_1, s_2)
	Stream Ops	*sop*	::=	+ | − | ∗ | / | % | ...

DLC IR for Running Knowledge Graph (KG) example:

```
...
04  dlc.for(str<i32> s_i = 0 to num_idxs) {
05    str<i32> s_idx = dlc.mem_str(idx_vec[i])
06    dlc.for(str<i32> s_j = 0 to emb_len) {
07      str<f32> s_v = dlc.mem_str(emb_tbl[idx,j])
08      dlc.callback() {
09        f32 i = dlc.to_val(s_i)
10        f32 j = dlc.to_val(s_j)
11        f32 v = dlc.to_val(s_v)
12        compute(i, j, v, res) }}}
```

The Decoupled Lookup-Compute (DLC) IR

Any CPU Statement	*STMT*	*Memory Reference*	*mref*
CPU variable	*var*	*Stream Variable*	*s*
Unsigned Integer	uint		

Control code	*For-Loop*	*FOR*	::=	for (*HEAD*) { *SDEC** *BODY* }
	For Header	*HEAD*	::=	*s* = *r* to *r* step uint
	Stream Declaration	*SDEC*	::=	*s* = *se*
	Loop Body	*BODY*	::=	*CALL* | *FOR*
Execute code	*Callback*	*CALL*	::=	callback { *TVAL** *STMT*$^+$ }
	Value Conversion	*TVAL*	::=	*var* = to_val(*s*)
	Loop Range	*r*	::=	*s* | *var* | uint
	Stream Expression	*se*	::=	*ms* | *as*
	Memory Stream	*ms*	::=	mem_str(*mref*[*indices**])
Access code	*Indices*	*indices*	::=	*s* | *var* | uint
	ALU Stream	*as*	::=	alu_str(*sop*, s_1, s_2)
	Stream Ops	*sop*	::=	+ | − | * | / | % | ...

DLC IR for Running Knowledge Graph (KG) example:

```
...
dlc.for(str<i32> s_i = 0 to num_idxs) {
  str<i32> s_idx = dlc.mem_str(idx_vec[i])
  dlc.for(str<i32> s_j = 0 to emb_len) {
    str<f32> s_v = dlc.mem_str(emb_tbl[idx,j])
    dlc.callback() {
      f32 i = dlc.to_val(s_i)
      f32 j = dlc.to_val(s_j)
      f32 v = dlc.to_val(s_v)
      compute(i, j, v, res) }}}
```

The Decoupled Lookup-Compute (DLC) IR

Any CPU Statement	*STMT*	*Memory Reference*	*mref*
CPU variable	*var*	*Stream Variable*	*s*
Unsigned Integer	uint		

Control code	*For-Loop*	*FOR*	::=	for (*HEAD*) { *SDEC** *BODY* }
	For Header	*HEAD*	::=	*s* = *r* to *r* step uint
	Stream Declaration	*SDEC*	::=	*s* = *se*
	Loop Body	*BODY*	::=	*CALL* | *FOR*
Execute code	*Callback*	*CALL*	::=	callback { *TVAL** *STMT*$^+$ }
Marshaling code	*Value Conversion*	*TVAL*	::=	*var* = to_val(*s*)
	Loop Range	*r*	::=	*s* | *var* | uint
	Stream Expression	*se*	::=	*ms* | *as*
	Memory Stream	*ms*	::=	mem_str(*mref*[*indices**])
Access code	*Indices*	*indices*	::=	*s* | *var* | uint
	ALU Stream	*as*	::=	alu_str(*sop*, s_1, s_2)
	Stream Ops	*sop*	::=	+ | − | * | / | % | ...

DLC IR for Running Knowledge Graph (KG) example:

```
...
dlc.for(str<i32> s_i = 0 to num_idxs) {
  str<i32> s_idx = dlc.mem_str(idx_vec[i])
  dlc.for(str<i32> s_j = 0 to emb_len) {
    str<f32> s_v = dlc.mem_str(emb_tbl[idx,j])
    dlc.callback() {
      f32 i = dlc.to_val(s_i)
      f32 j = dlc.to_val(s_j)
      f32 v = dlc.to_val(s_v)
      compute(i, j, v, res) }}}
```

Then code gets generated from the DLC IR

DLC IR for KG:

```
…
dlc.for(str<i32> s_i = 0 to num_idxs) {
  str<i32> s_idx = dlc.mem_str(idx_vec[i])
  dlc.for(str<i32> s_j = 0 to emb_len) {
    str<f32> s_v = dlc.mem_str(emb_tbl[idx,j])
    dlc.callback() {
      f32 i = dlc.to_val(s_i)
      f32 j = dlc.to_val(s_j)
      f32 v = dlc.to_val(s_v)
      compute(i, j, v, res) }}}
```

Then code gets generated from the DLC IR

DLC IR for KG:

```
…
dlc.for(str<i32> s_i = 0 to num_idxs) {
  str<i32> s_idx = dlc.mem_str(idx_vec[i])
  dlc.for(str<i32> s_j = 0 to emb_len) {
    str<f32> s_v = dlc.mem_str(emb_tbl[idx,j])
    dlc.callback() {
      f32 i = dlc.to_val(s_i)
      f32 j = dlc.to_val(s_j)
      f32 v = dlc.to_val(s_v)
      compute(i, j, v, res) }}}
```

Dataflow lookup code:

```
i_tr = loop_tr(0, num_idxs)
i_str = i_tr.mem_str(idx_vec, [i_tr.ite])
e_tr = loop_tr(0, emb_len)
e_str = i_tr.mem_str(emb_tbl, [i_str,e_tr.ite])
serialize(i_tr.ite, e_tr.ite, e_str)
```

Compute code:

```
deserialize(i, j, v)
compute(i, j, v, res)
```

Lowering from imperative loops to the DLC IR

DLC IR:

```
…
dlc.for(str<i32> s_i = 0 to num_idxs) {
  str<i32> s_idx = dlc.mem_str(idx_vec[i])
  dlc.for(str<i32> s_j = 0 to emb_len) {
    str<f32> s_v = dlc.mem_str(emb_tbl[idx,j])
    dlc.callback() {
      f32 i = dlc.to_val(s_i)
      f32 j = dlc.to_val(s_j)
      f32 v = dlc.to_val(s_v)
      compute(i, j, v, res) }}}
```

Lowering from imperative loops to the DLC IR

Imperative MLIR:

```
…
for i from 0 to num_idxs:
  idx = load(idx_vec, [i])
  for j from 0 to emb_len:
    v = load(emb_tbl, [idx,j])
    compute(i, j, v, res)
```

DLC IR:

```
…
dlc.for(str<i32> s_i = 0 to num_idxs) {
  str<i32> s_idx = dlc.mem_str(idx_vec[i])
  dlc.for(str<i32> s_j = 0 to emb_len) {
    str<f32> s_v = dlc.mem_str(emb_tbl[idx,j])
    dlc.callback() {
      f32 i = dlc.to_val(s_i)
      f32 j = dlc.to_val(s_j)
      f32 v = dlc.to_val(s_v)
      compute(i, j, v, res) }}}
```

Lowering from imperative loops to the DLC IR

Imperative MLIR:

```
…
for i from 0 to num_idxs:
  idx = load(idx_vec, [i])
  for j from 0 to emb_len:
    v = load(emb_tbl, [idx,j])
    compute(i, j, v, res)
```

DLC IR:

```
…
dlc.for(str<i32> s_i = 0 to num_idxs) {
  str<i32> s_idx = dlc.mem_str(idx_vec[i])
  dlc.for(str<i32> s_j = 0 to emb_len) {
    str<f32> s_v = dlc.mem_str(emb_tbl[idx,j])
    dlc.callback() {
      f32 i = dlc.to_val(s_i)
      f32 j = dlc.to_val(s_j)
      f32 v = dlc.to_val(s_v)
      compute(i, j, v, res) }}}
```

Lowering from imperative loops to the DLC IR

Imperative MLIR:

```
…
for i from 0 to num_idxs:
  idx = load(idx_vec, [i])
  for j from 0 to emb_len:
    v = load(emb_tbl, [idx,j])
    compute(i, j, v, res)
```

DLC IR:

```
…
dlc.for(str<i32> s_i = 0 to num_idxs) {
  str<i32> s_idx = dlc.mem_str(idx_vec[i])
  dlc.for(str<i32> s_j = 0 to emb_len) {
    str<f32> s_v = dlc.mem_str(emb_tbl[idx,j])
    dlc.callback() {
      f32 i = dlc.to_val(s_i)
      f32 j = dlc.to_val(s_j)
      f32 v = dlc.to_val(s_v)
      compute(i, j, v, res) }}}
```

Lowering from imperative loops to the DLC IR

Imperative MLIR:

```
…
for i from 0 to num_idxs:
  idx = load(idx_vec, [i])
  for j from 0 to emb_len:
    v = load(emb_tbl, [idx,j])
    compute(i, j, v, res)
```

DLC IR:

```
…
dlc.for(str<i32> s_i = 0 to num_idxs) {
  str<i32> s_idx = dlc.mem_str(idx_vec[i])
  dlc.for(str<i32> s_j = 0 to emb_len) {
    str<f32> s_v = dlc.mem_str(emb_tbl[idx,j])
    dlc.callback() {
      f32 i = dlc.to_val(s_i)
      f32 j = dlc.to_val(s_j)
      f32 v = dlc.to_val(s_v)
      compute(i, j, v, res) }}}
```

Lowering from imperative loops to the DLC IR

Imperative MLIR:

```
…
for i from 0 to num_idxs:
  idx = load(idx_vec, [i])
  for j from 0 to emb_len:
    v = load(emb_tbl, [idx,j])
    compute(i, j, v, res)
```

DLC IR:

```
…
dlc.for(str<i32> s_i = 0 to num_idxs) {
  str<i32> s_idx = dlc.mem_str(idx_vec[i])
  dlc.for(str<i32> s_j = 0 to emb_len) {
    str<f32> s_v = dlc.mem_str(emb_tbl[idx,j])
    dlc.callback() {
      f32 i = dlc.to_val(s_i)
      f32 j = dlc.to_val(s_j)
      f32 v = dlc.to_val(s_v)
      compute(i, j, v, res) }}}
```

Lowering from imperative loops to the DLC IR

Imperative MLIR:

```
…
for i from 0 to num_idxs:
  idx = load(idx_vec, [i])
  for j from 0 to emb_len:
    v = load(emb_tbl, [idx,j])
    compute(i, j, v, res)
```

DLC IR:

```
…
dlc.for(str<i32> s_i = 0 to num_idxs) {
  str<i32> s_idx = dlc.mem_str(idx_vec[i])
  dlc.for(str<i32> s_j = 0 to emb_len) {
    str<f32> s_v = dlc.mem_str(emb_tbl[idx,j])
    dlc.callback() {
      f32 i = dlc.to_val(s_i)
      f32 j = dlc.to_val(s_j)
      f32 v = dlc.to_val(s_v)
      compute(i, j, v, res) }}}
```

A key optimization is auto-vectorization

DLC IR:

```
…
dlc.for(str<i32> s_i = 0 to num_idxs) {
  str<i32> s_idx = dlc.mem_str(idx_vec[i])
  dlc.for(str<i32> s_j = 0 to emb_len) {
    str<f32> s_v = dlc.mem_str(emb_tbl[idx,j])
    dlc.callback() {
      f32 i = dlc.to_val(s_i)
      f32 j = dlc.to_val(s_j)
      f32 v = dlc.to_val(s_v)
      compute(i, j, v, res) }}}
```

A key optimization is auto-vectorization

DLC IR:

```
…
dlc.for(str<i32> s_i = 0 to num_idxs) {
  str<i32> s_idx = dlc.mem_str(idx_vec[i])
  dlc.for(str<i32> s_j = 0 to emb_len) {
    str<f32> s_v = dlc.mem_str(emb_tbl[idx,j])
    dlc.callback() {
      f32 i = dlc.to_val(s_i)
      f32 j = dlc.to_val(s_j)
      f32 v = dlc.to_val(s_v)
      compute(i, j, v, res) }}}
```

Vectorized DLC IR:

```
…
dlc.for(str<i32> s_i = 0 to num_idxs) {
  str<i32> s_idx = dlc.mem_str(idx_vec[i])
  dlcv.for<vlen>(str<i32> s_j = 0 to emb_len, msk) {
    str<f32> s_v = dlc.mem_str<vlen>(emb_tbl[idx,j],msk)
    dlc.callback() {
      f32 i = dlc.to_val(s_i)
      f32 j = dlcv.to_val<vlen>(s_j)[0]
      f32 v = dlcv.to_val<vlen>(s_v)
      compute<vlen>(i, j, v, res) }}}
```

A key optimization is auto-vectorization

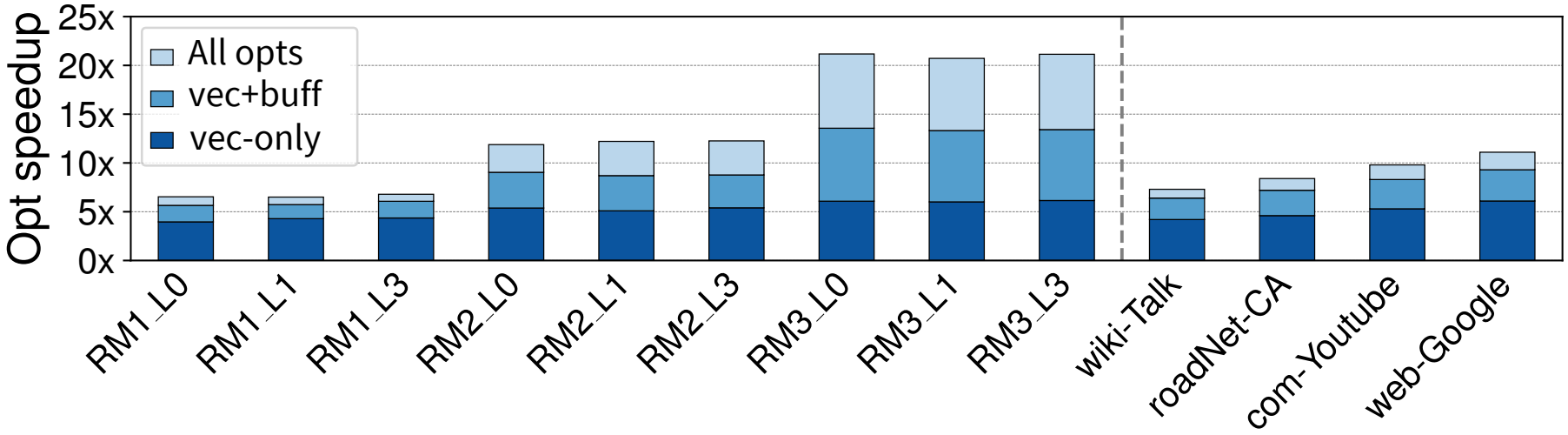
DLC IR:

```
…
dlc.for(str<i32> s_i = 0 to num_idxs) {
  str<i32> s_idx = dlc.mem_str(idx_vec[i])
  dlc.for(str<i32> s_j = 0 to emb_len) {
    str<f32> s_v = dlc.mem_str(emb_tbl[idx,j])
    dlc.callback() {
      f32 i = dlc.to_val(s_i)
      f32 j = dlc.to_val(s_j)
      f32 v = dlc.to_val(s_v)
      compute(i, j, v, res) }}}
```

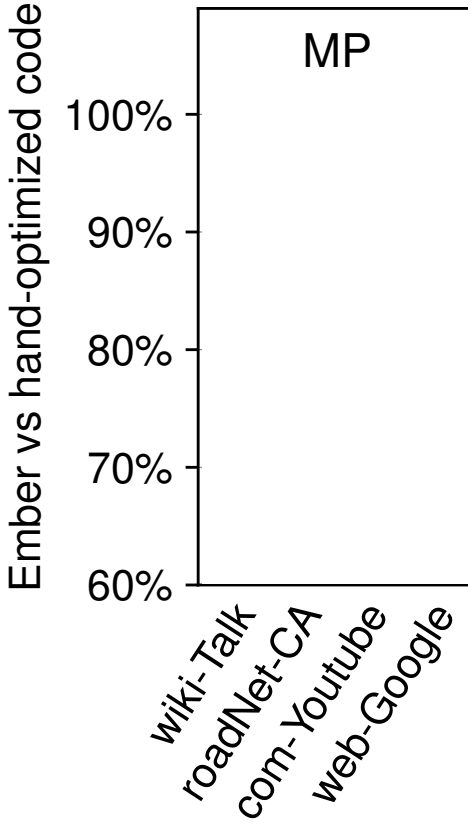
Vectorized DLC IR:

```
…
dlc.for(str<i32> s_i = 0 to num_idxs) {
  str<i32> s_idx = dlc.mem_str(idx_vec[i])
  dlcv.for<vlen>(str<i32> s_j = 0 to emb_len, msk) {
    str<f32> s_v = dlc.mem_str<vlen>(emb_tbl[idx,j],msk)
    dlc.callback() {
      f32 i = dlc.to_val(s_i)
      f32 j = dlcv.to_val<vlen>(s_j)[0]
      f32 v = dlcv.to_val<vlen>(s_v)
      compute<vlen>(i, j, v, res) }}}
```

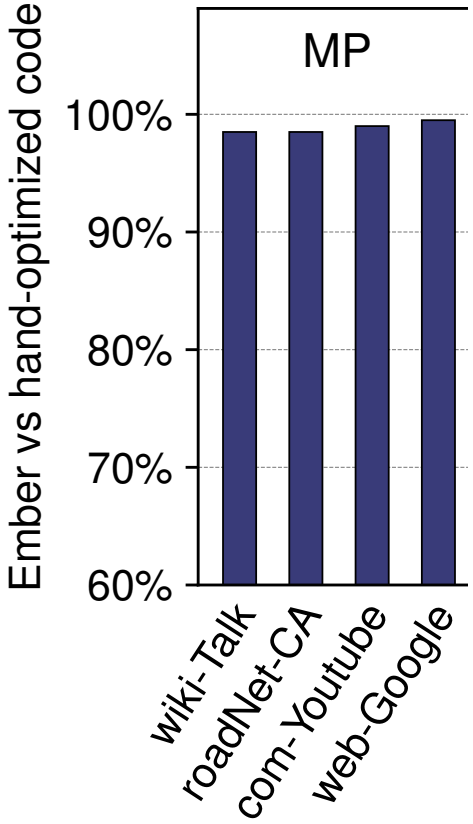
Optimizations are important for performance



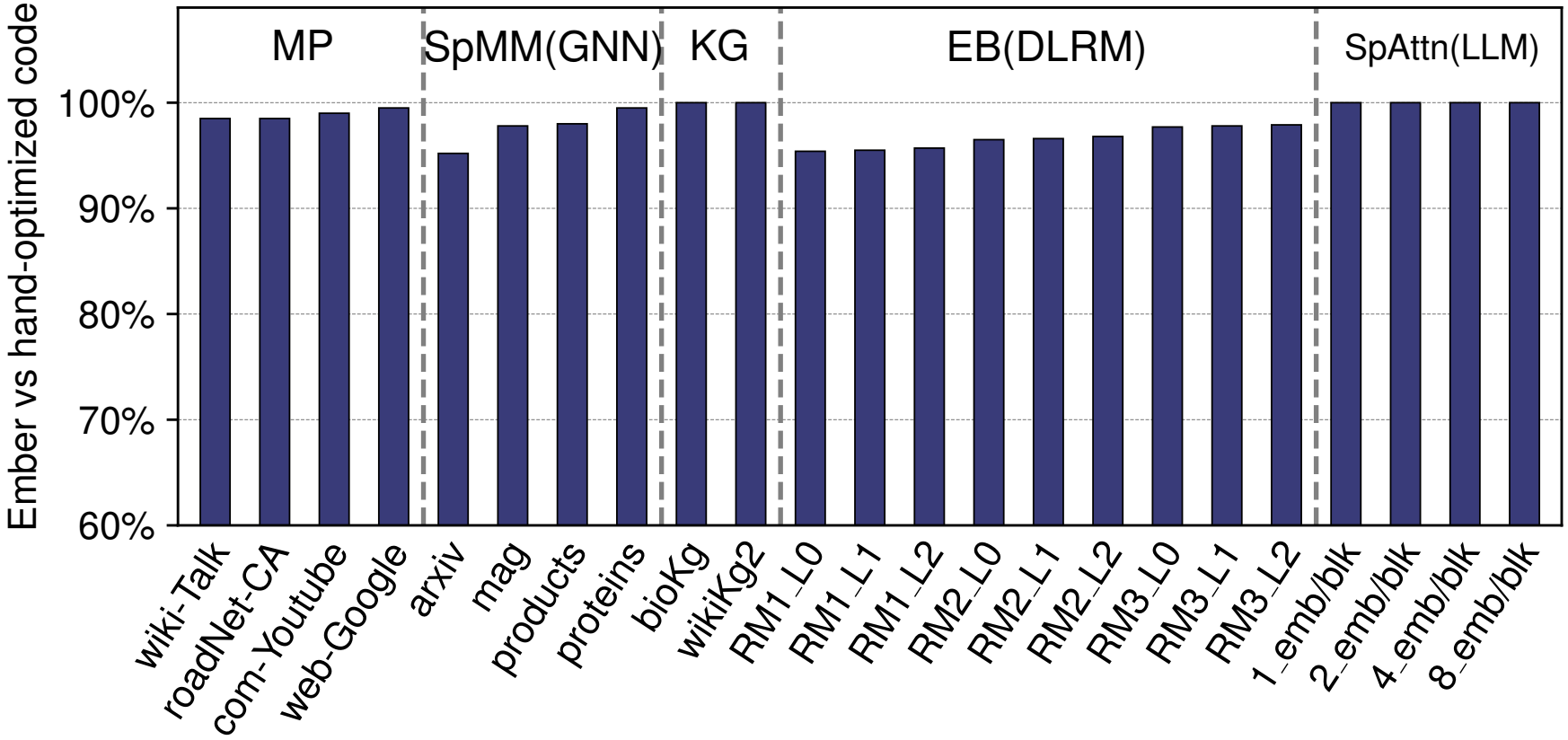
Ember vs hand-optimized code



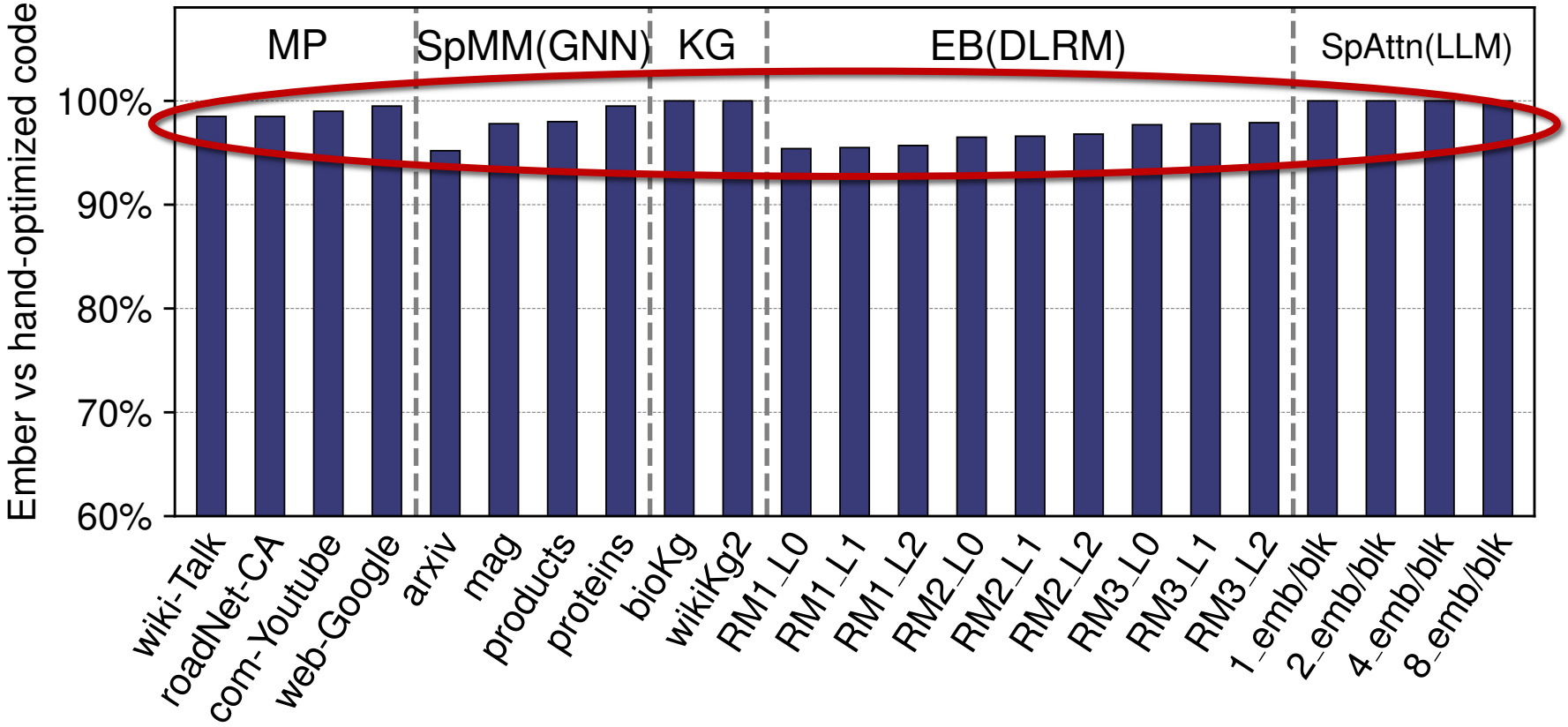
Ember vs hand-optimized code



Ember vs hand-optimized code

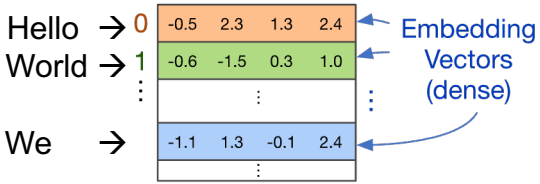


Ember vs hand-optimized code

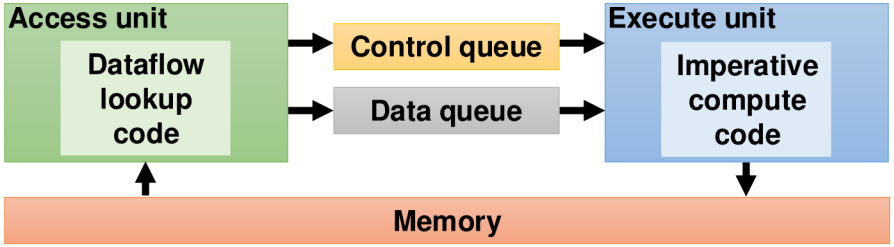


Conclusion

marco.siracusa@bsc.es



Embedding Operations in PyTorch



This work is supported by the project PID2023-146511NB-I00 funded by the Spanish Ministry of Science, Innovation and Universities MCIU /AEI /10.13039/501100011033 and EU ERDF.

Barcelona Supercomputing Center
Centro Nacional de Supercomputación

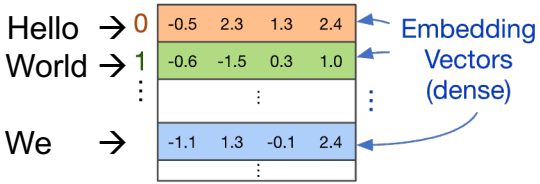
Stanford | ENGINEERING
Computer Science

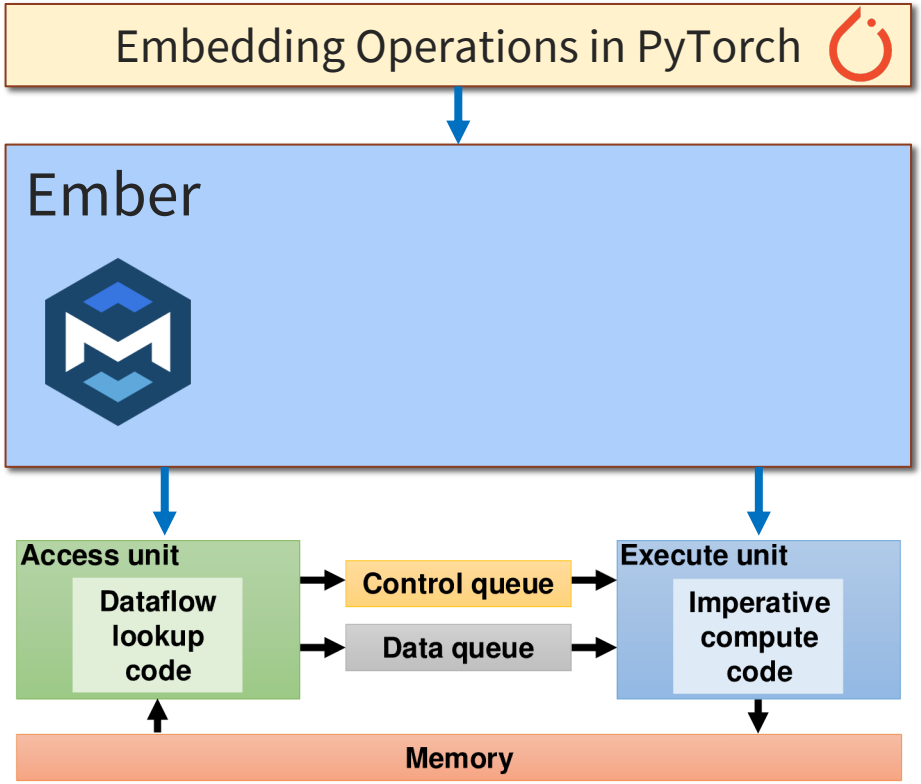
arm

SAMSUNG SAIT

Conclusion

marco.siracusa@bsc.es





This work is supported by the project PID2023-146511NB-I00 funded by the Spanish Ministry of Science, Innovation and Universities MCIU /AEI /10.13039/501100011033 and EU ERDF.

Barcelona Supercomputing Center
Centro Nacional de Supercomputación

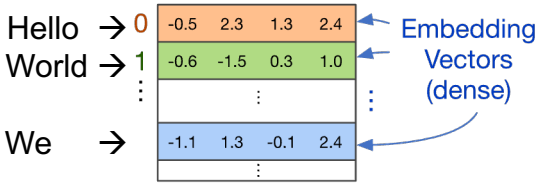
Stanford | ENGINEERING
Computer Science

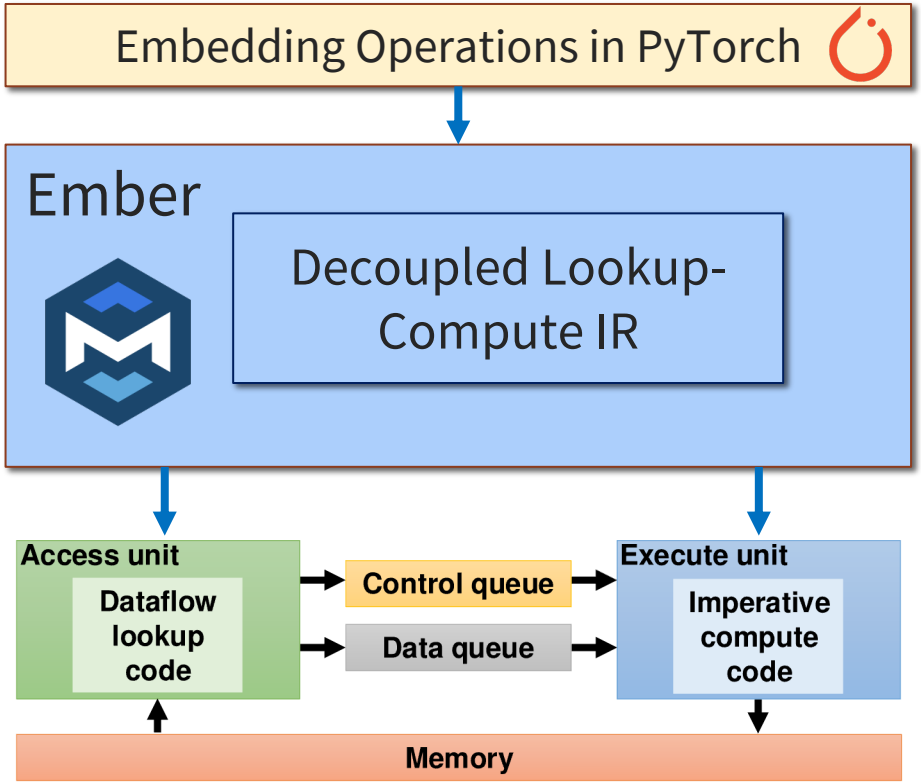
arm

SAMSUNG SAIT

Conclusion

marco.siracusa@bsc.es





This work is supported by the project PID2023-146511NB-I00 funded by the Spanish Ministry of Science, Innovation and Universities MCIU /AEI /10.13039/501100011033 and EU ERDF.

Barcelona Supercomputing Center
Centro Nacional de Supercomputación

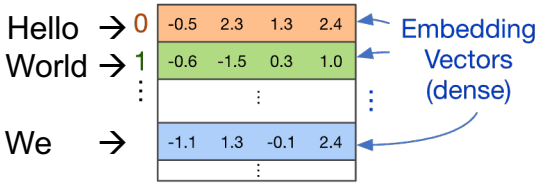
Stanford | ENGINEERING
Computer Science

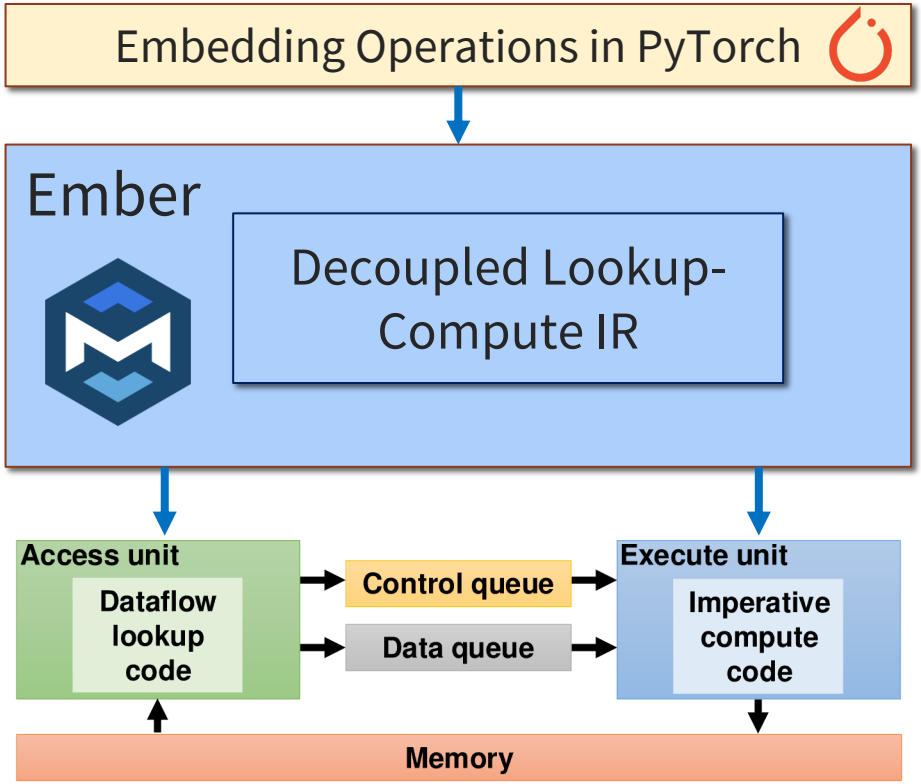
arm

SAMSUNG SAIT

Conclusion

marco.siracusa@bsc.es





Nvidia's GPU TMA

Google's TPU SparseCore

Dataflow Accelerators

…and more

This work is supported by the project PID2023-146511NB-I00 funded by the Spanish Ministry of Science, Innovation and Universities MCIU /AEI /10.13039/501100011033 and EU ERDF.

Barcelona Supercomputing Center
Centro Nacional de Supercomputación

Stanford | ENGINEERING
Computer Science

arm

SAMSUNG SAIT

Code generation from the DLC IR to TMU-CPU code

DLC IR:

```
…
dlc.for(str<i32> s_i = 0 to num_idxs) {
  str<i32> s_idx = dlc.mem_str(idx_vec[i])
  dlc.for(str<i32> s_j = 0 to emb_len) {
    str<f32> s_v = dlc.mem_str(emb_tbl[idx,j])
    dlc.callback() {
      f32 i = dlc.to_val(s_i)
      f32 j = dlc.to_val(s_j)
      f32 v = dlc.to_val(s_v)
      compute(i, j, v, res) }}}
```

DAE code to TMU-CPU code lowering:

Code generation from the DLC IR to TMU-CPU code

DLC IR:

```
…
dlc.for(str<i32> s_i = 0 to num_idxs) {
  str<i32> s_idx = dlc.mem_str(idx_vec[i])
  dlc.for(str<i32> s_j = 0 to emb_len) {
    str<f32> s_v = dlc.mem_str(emb_tbl[idx,j])
    dlc.callback() {
      f32 i = dlc.to_val(s_i)
      f32 j = dlc.to_val(s_j)
      f32 v = dlc.to_val(s_v)
      compute(i, j, v, res) }}}
```

TMU code:

```
…
i_tr = loop_tr(0, num_idxs)
i_str = i_tr.mem_str(idx_vec, [i_tr.ite])
e_tr = loop_tr(0, emb_len)
e_str = i_tr.mem_str(emb_tbl, [i_str, e_tr.ite])
serialize(i_tr.ite, e_tr.ite, e_str)
```

DAE code to TMU-CPU code lowering:

- For the access code, **transform**

Code generation from the DLC IR to TMU-CPU code

DLC IR:

```
…
dlc.for(str<i32> s_i = 0 to num_idxs) {
  str<i32> s_idx = dlc.mem_str(idx_vec[i])
  dlc.for(str<i32> s_j = 0 to emb_len) {
    str<f32> s_v = dlc.mem_str(emb_tbl[idx,j])
    dlc.callback() {
      f32 i = dlc.to_val(s_i)
      f32 j = dlc.to_val(s_j)
      f32 v = dlc.to_val(s_v)
      compute(i, j, v, res) }}}
```

TMU code:

```
…
i_tr = loop_tr(0, num_idxs)
i_str = i_tr.mem_str(idx_vec, [i_tr.ite])
e_tr = loop_tr(0, emb_len)
e_str = i_tr.mem_str(emb_tbl, [i_str, e_tr.ite])
serialize(i_tr.ite, e_tr.ite, e_str)
```

CPU code:

```
…
while(…)
  deserialize(i, j, v)
  compute(i, j, v, res)
```

DAE code to TMU-CPU code lowering:

- For the access code, **transform**
  - `dlc.for` → TMU iteration primitives

Code generation from the DLC IR to TMU-CPU code

DLC IR:

```
…
dlc.for(str<i32> s_i = 0 to num_idxs) {
  str<i32> s_idx = dlc.mem_str(idx_vec[i])
  dlc.for(str<i32> s_j = 0 to emb_len) {
    str<f32> s_v = dlc.mem_str(emb_tbl[idx,j])
    dlc.callback() {
      f32 i = dlc.to_val(s_i)
      f32 j = dlc.to_val(s_j)
      f32 v = dlc.to_val(s_v)
      compute(i, j, v, res) }}}
```

TMU code:

```
…
i_tr = loop_tr(0, num_idxs)
i_str = i_tr.mem_str(idx_vec, [i_tr.ite])
e_tr = loop_tr(0, emb_len)
e_str = i_tr.mem_str(emb_tbl, [i_str, e_tr.ite])
serialize(i_tr.ite, e_tr.ite, e_str)
```

DAE code to TMU-CPU code lowering:

- For the access code, **transform**
  - `dlc.for` → TMU iteration primitives
  - `dlcmem_str` → TMU memory streams
  - `dlc.alu_str` → TMU arithmetic streams

Code generation from the DLC IR to TMU-CPU code

DLC IR:

```
…
dlc.for(str<i32> s_i = 0 to num_idxs) {
  str<i32> s_idx = dlc.mem_str(idx_vec[i])
  dlc.for(str<i32> s_j = 0 to emb_len) {
    str<f32> s_v = dlc.mem_str(emb_tbl[idx,j])
    dlc.callback() {
      f32 i = dlc.to_val(s_i)
      f32 j = dlc.to_val(s_j)
      f32 v = dlc.to_val(s_v)
      compute(i, j, v, res) }}}
```

TMU code:

```
…
i_tr = loop_tr(0, num_idxs)
i_str = i_tr.mem_str(idx_vec, [i_tr.ite])
e_tr = loop_tr(0, emb_len)
e_str = i_tr.mem_str(emb_tbl, [i_str, e_tr.ite])
serialize(i_tr.ite, e_tr.ite, e_str)
```

CPU code:

```
…
while(…)
  deserialize(i, j, v)
  compute(i, j, v, res)
```

DAE code to TMU-CPU code lowering:

- For the access code, **transform**
  - `dlc.for` → TMU iteration primitives
  - `dlcmem_str` → TMU memory streams
  - `dlc.alu_str` → TMU arithmetic streams
- For the compute code, **transform**

Lowering – DLC to decoupled TMU-CPU

DLC IR:

```
…
dlc.for(str<i32> s_i = 0 to num_idxs) {
  str<i32> s_idx = dlc.mem_str(idx_vec[i])
  dlc.for(str<i32> s_j = 0 to emb_len) {
    str<f32> s_v = dlc.mem_str(emb_tbl[idx,j])
    dlc.callback() {
      f32 i = dlc.to_val(s_i)
      f32 j = dlc.to_val(s_j)
      f32 v = dlc.to_val(s_v)
      compute(i, j, v, res) }}}
```

TMU code:

```
…
i_tr = loop_tr(0, num_idxs)
i_str = i_tr.mem_str(idx_vec, [i_tr.ite])
e_tr = loop_tr(0, emb_len)
e_str = i_tr.mem_str(emb_tbl, [i_str, e_tr.ite])
serialize(i_tr.ite, e_tr.ite, e_str)
```

CPU code:

```
…
while(…)
  deserialize(i, j, v)
  compute(i, j, v, res)
```

DAE code to TMU-CPU code lowering:

- For the access code, **transform**
  - `dlc.for` → TMU iteration primitives
  - `dlcmem_str` → TMU memory streams
  - `dlc.alu_str` → TMU arithmetic streams
- For the compute code, **transform**
  - `dlc.callback` ops → while-loop and if-then-else chains

Code generation from the DLC IR to TMU-CPU code

DLC IR:

```
…
dlc.for(str<i32> s_i = 0 to num_idxs) {
  str<i32> s_idx = dlc.mem_str(idx_vec[i])
  dlc.for(str<i32> s_j = 0 to emb_len) {
    str<f32> s_v = dlc.mem_str(emb_tbl[idx,j])
    dlc.callback() {
      f32 i = dlc.to_val(s_i)
      f32 j = dlc.to_val(s_j)
      f32 v = dlc.to_val(s_v)
      compute(i, j, v, res) }}}
```

TMU code:

```
…
i_tr = loop_tr(0, num_idxs)
i_str = i_tr.mem_str(idx_vec, [i_tr.ite])
e_tr = loop_tr(0, emb_len)
e_str = i_tr.mem_str(emb_tbl, [i_str, e_tr.ite])
serialize(i_tr.ite, e_tr.ite, e_str)
```

CPU code:

```
…
while(…)
  deserialize(i, j, v)
  compute(i, j, v, res)
```

DAE code to TMU-CPU code lowering:

- For the access code, **transform**
  - `dlc.for` → TMU iteration primitives
  - `dlcmem_str` → TMU memory streams
  - `dlc.alu_str` → TMU arithmetic streams
- For the compute code, **transform**
  - `dlc.callback` ops → while-loop and if-then-else chains
  - `dlc.to_val` ops → marshaling operations

Code generation from the DLC IR to TMU-CPU code

DLC IR:

```
…
dlc.for(str<i32> s_i = 0 to num_idxs) {
  str<i32> s_idx = dlc.mem_str(idx_vec[i])
  dlc.for(str<i32> s_j = 0 to emb_len) {
    str<f32> s_v = dlc.mem_str(emb_tbl[idx,j])
    dlc.callback() {
      f32 i = dlc.to_val(s_i)
      f32 j = dlc.to_val(s_j)
      f32 v = dlc.to_val(s_v)
      compute(i, j, v, res) }}}
```

TMU code:

```
…
i_tr = loop_tr(0, num_idxs)
i_str = i_tr.mem_str(idx_vec, [i_tr.ite])
e_tr = loop_tr(0, emb_len)
e_str = i_tr.mem_str(emb_tbl, [i_str, e_tr.ite])
serialize(i_tr.ite, e_tr.ite, e_str)
```

CPU code:

```
…
while(…)
  deserialize(i, j, v)
  compute(i, j, v, res)
```

DAE code to TMU-CPU code lowering:

- For the access code, **transform**
  - `dlc.for` → TMU iteration primitives
  - `dlcmem_str` → TMU memory streams
  - `dlc.alu_str` → TMU arithmetic streams
- For the compute code, **transform**
  - `dlc.callback` ops → while-loop and if-then-else chains
  - `dlc.to_val` ops → marshaling operations